
Transformadores

PID_00290602

Jordi de la Torre Gallart

Tiempo mínimo de dedicación recomendado: 5 horas



Jordi de la Torre Gallart

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Jordi Casas Roma

Cómo citar este recurso de aprendizaje con el estilo Harvard:

De la Torre Gallart, J. (2024). *Transformadores*. [Recurso de aprendizaje textual]. 1.^a ed. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: febrero 2024
© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)
Av. Tibidabo, 39-43, 08035 Barcelona
Autoría: Jordi de la Torre Gallart
Producción: FUOC
Todos los derechos reservados

Ninguna parte de esta publicación, incluido el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma, ni por ningún medio, sea este eléctrico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita del titular de los derechos.

Índice general

Introducción	5
Objetivos	6
1 Introducción a los transformadores	7
1.1 Evolución histórica de los mecanismos de atención	8
1.2 Desafíos asociados a la arquitectura de las redes recurrentes ..	9
2 Arquitectura de los transformadores	11
2.1 El mecanismo de autoatención	11
2.1.1 Definición formal	11
2.1.2 Definición conceptual: una analogía con las bases de datos	14
2.1.3 Autoatención múltiple	15
2.2 Transformadores	16
2.2.1 Notación	17
2.2.2 Elementos constitutivos del transformador	18
2.3 Tipos de transformadores	23
2.3.1 Transformadores secuencia a secuencia	23
2.3.2 Transformadores bidireccionales o de autocodificación	26
2.3.3 Transformadores autorregresivos	29
2.4 Transformadores para procesamiento del lenguaje natural (NLP)	33
2.4.1 Desarrollos previos: <i>embeddings</i>	33
2.4.2 Inicios de los transformadores para NLP	34
2.5 Transformadores para Visión (ViT)	35
2.5.1 Clasificación	36
2.5.2 Detección	38
2.5.3 Segmentación	41
2.6 Transformadores para audio	43
2.6.1 Señales de audio, tratamiento y <i>tokenización</i>	43
2.6.2 Ejemplos de transformadores de audio	44
2.7 Transformadores multimodales	45
2.7.1 Entradas multimodales	46
2.7.2 Variantes de autoatención en un contexto multimodal	46
2.7.3 Arquitecturas multimodales	50

2.8	Aplicaciones de los transformadores multimodales	50
Resumen		52
Glosario		53
Notación		54

Introducción

Los **transformadores** son una arquitectura de red neuronal diseñada originalmente para la transformación de datos secuencia a secuencia, como alternativa a las redes neuronales recurrentes. Su ámbito de aplicación actual es más general. Se ha convertido en una arquitectura generalista capaz de competir e incluso superar a otro tipo de redes especializadas como las recurrentes y las convolucionales. Su uso actual se extiende a tratamiento de texto, sonido, imagen, aprendizaje por refuerzo, entre otros, así como a aplicaciones con datos de entrada heterogéneos que usan distintas clases de datos de entrada.

La mayoría de las técnicas que estudiaremos en este módulo han sido desarrolladas en los últimos años y, por tanto, están sometidas a evolución. El objetivo principal de este módulo es proveer al lector del contexto necesario para que sea capaz de comprender los artículos de investigación más recientes relacionados con el tema. Los distintos apartados se centran en introducir los fundamentos matemáticos y algorítmicos de todos los elementos integrantes de las redes de transformadores, para así comprender el funcionamiento de esta arquitectura.

Propuestas inicialmente en Vaswani y otros (2017), desde su introducción se convirtieron en el estado del arte para el tratamiento de los problemas típicos de procesamiento del lenguaje natural (NLP). Posteriormente, su aplicación se ha ido ampliando a otros ámbitos, convirtiéndose en una herramienta principal para resolver problemas más allá del NLP.

El módulo fundamental y distintivo de la arquitectura es un sistema de **auto-atención**, esto es, la denominada atención a la propia secuencia. Este mecanismo de autoatención deriva del sistema de **atención** introducido en los años anteriores como medio para intentar resolver algunos problemas inherentes al procesamiento de largas secuencias.

En este módulo didáctico introduciremos la arquitectura de los transformadores (*transformers*). Iniciaremos el módulo realizando una mirada histórica a la casuística que llevó a la introducción y posterior evolución de los mecanismos de atención hasta llegar a una solución estable basada en los transformadores.

A continuación estudiaremos los distintos componentes que conforman esta arquitectura, así como las distintas variaciones que pueden existir.

Finalmente, veremos algunas de las aplicaciones para estos modelos, empezando por las basadas en procesamiento del lenguaje y por la clasificación de imágenes.

Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Conocer la casuística que llevó a la necesidad de plantear una arquitectura alternativa para tratar las tareas de transformación de **secuencia a secuencia**.
2. Conocer los módulos típicos que integran la arquitectura interna de los modelos basados en los transformadores, así como las distintas variaciones.
3. Conocer cómo aplicar estos modelos para la solución de problemas de procesamiento de lenguaje natural.
4. Conocer cómo aplicar estos modelos para la solución de problemas de procesamiento de imagen.
5. Conocer las posibilidades de estos modelos para la resolución de problemas más complejos que utilicen entradas de distinta naturaleza.

1. Introducción a los transformadores

Las redes neuronales completamente conectadas son aproximadores universales. Ya en 1989, Kornik y otros demostraron que a partir de un conjunto definido de nodos de entrada, con únicamente una capa oculta y con suficientes nodos, es posible aproximar cualquier función continua con la precisión deseada (Hornik y otros, 1989).

A pesar de esta realidad teórica matemática, la implementación práctica es más compleja y hace que sea difícil encontrar el número de nodos y valor de los pesos adecuados para conseguir la aproximación deseada.

De cara a facilitar el proceso de optimización de las redes, históricamente, se han utilizado priores estructurales que permiten restringir el espacio de optimización a un subconjunto más acotado a los valores esperados. Así, en el caso de imagen, por ejemplo, las redes convolucionales explotan en su beneficio predictivo la propiedad inherente de nuestro entorno de que las regularidades estadísticas útiles para realizar una predicción suelen estar situadas en un entorno de proximidad espacial. Por otro lado, las redes recurrentes, optimizadas para el tratamiento de datos secuenciales, explotan la propiedad de localidad temporal, esto es, que en cierto modo la información más relevante para predecir un evento suele estar en el entorno más próximo. Estas arquitecturas, por tanto, están optimizadas para sacar partido de las regularidades espaciales y temporales presentes en las señales sensoriales de interés predictivo típicas de nuestro mundo (Ioannou, 2018).

A pesar de que estos priores estructurales suelen facilitar el proceso de aprendizaje, en algunos casos pueden suponer una desventaja. Las redes neuronales recurrentes se encuentran a menudo con problemas relacionados con la resolución de dependencias de largo alcance, que las arquitecturas existentes no son capaces de resolver. Limitaciones en la memoria interna disponible, así como dificultades con la propagación de gradientes a través de una larga secuencia, pueden ser causa de un rendimiento insatisfactorio en las funciones encomendadas.

Los modelos basados en mecanismos de atención nacen precisamente con el objetivo de abordar esas dependencias mal resueltas, como un mecanismo que pretende modelizar matemáticamente la enfatización de los atributos relevantes a la hora de hacer una predicción.

1.1 Evolución histórica de los mecanismos de atención

El objetivo de este subapartado no es realizar una presentación exhaustiva de todas las modalidades de mecanismos de atención existentes, pero sí introducir el concepto y para, posteriormente, presentar la evolución que dio lugar al sistema de **autoatención** utilizado en los transformadores.

La importancia de la arquitectura de los transformadores se puede equiparar con la que supuso en su momento la propuesta de otras invenciones anteriores mencionadas en este módulo, como las redes convolucionales y las redes recurrentes LSTM. Como ya hemos dicho con anterioridad, supone una mejora sustancial respecto a sus predecesoras, convirtiéndose en una arquitectura generalista que permite no únicamente tratar sino también integrar en la predicción de forma elegante datos de naturaleza diversa.

El hito más importante de la clase de redes neuronales basadas en mecanismos de atención ha sido la eliminación de la necesidad de la recurrencia para el tratamiento de datos secuenciales, sustituyendo la representación de las dependencias de un estado interno de la máquina a un método de atender en paralelo a las partes relevantes del conjunto de la secuencia. Básicamente, convertir la dependencia temporal en una espacial, codificando la temporalidad como una parte adicional de la entrada.

La evolución histórica de las ideas que llevaron al estado del arte actual en lo referente a atención, empieza alrededor de 2014. El interés por dotar de elementos explicativos a los modelos predictivos, llevó al diseño de sistemas basados en atención como modo de identificar los objetos presentes en imágenes que tenían más importancia para la realización de las predicciones de la red neuronal. En 2015, se propone la utilización de estos mecanismos en una aplicación de procesamiento del lenguaje natural (NLP), concretamente para la traducción de texto (Bahdanau y otros, 2014). Hasta ese momento se habían utilizado arquitecturas codificador-decodificador en las que la primera parte de la red se encargaba de codificar en un vector interno la frase de entrada en el idioma original y a continuación el decodificador la traducía al lenguaje objetivo. Los autores de esta nueva propuesta conjeturaron y demostraron experimentalmente que la utilización de información de los estados internos del codificador, y no únicamente del vector interno final, mejoraba los resultados de rendimiento.

A partir de esa primera propuesta se genera una nueva corriente de investigación en ese sentido, que lleva a que en 2017 se presente un artículo que cambió definitivamente el paradigma de diseño hacia una nueva arquitectura. En el artículo «Attention is all you need» (Vaswani y otros, 2017), se presenta una propuesta radical en la que se elimina totalmente la recurrencia, para diseñar una red formada por la combinación de módulos de **autoatención**, normalización y capas completamente conectadas, denominada por los auto-

res red de **transformadores**, consiguiendo resultados al nivel del estado del arte y eliminando las complejidades asociadas al entrenamiento de las redes neuronales recurrentes.

1.2 Desafíos asociados a la arquitectura de las redes recurrentes

Las redes recurrentes surgieron como una forma de abordar el tratamiento de los datos secuenciales. Este tipo de arquitectura se adapta a las características de estos datos, permitiendo la entrada en el orden establecido por la secuencia. Conceptualmente, la red es capaz de mantener un estado interno que recoge las dependencias respecto a la entrada. Estas pueden ser de corto, medio o largo alcance. Idealmente, la red debería ser capaz de recordar y establecer las relaciones entre esas dependencias de distinta naturaleza para predecir salidas futuras.

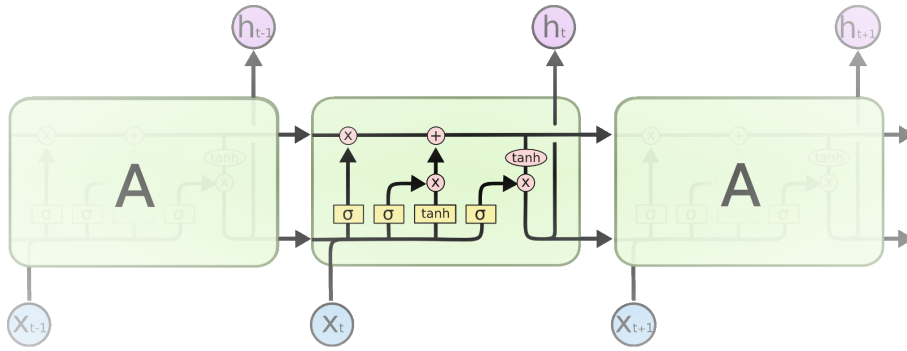
Este tipo de redes se utiliza como módulo dentro de modelos para abordar problemas de los siguientes tipos:

- 1) **Modelos vector-secuencia.** A partir de un vector de tamaño fijo de entrada se genera una secuencia de dimensión arbitraria. Un ejemplo sería el etiquetado de imágenes, donde la entrada es una imagen y la salida un texto explicativo.
- 2) **Modelos secuencia-vector.** La entrada es un vector de dimensión arbitraria y la salida uno de dimensión fija. Un ejemplo sería el análisis de sentimiento, donde a partir de una secuencia de entrada de dimensión variable se genera un valor binario que indica si el sentimiento de esa secuencia es positivo o negativo.
- 3) **Modelos secuencia-secuencia.** Tanto la entrada como la salida son secuencias de dimensión variable. Un ejemplo sería la traducción de un texto de un idioma a otro distinto.

La experiencia práctica demuestra que las arquitecturas basadas en redes neuronales recurrentes diseñadas para abordar estos problemas no únicamente requieren más épocas de entrenamiento que las redes convolucionales, sino que presentan una serie de inconvenientes relacionados con la alta profundidad de las redes desplegadas, como la dificultad para resolver las dependencias de largo alcance, explosión y desvanecimiento de gradientes y dificultad para la programación en paralelo de la computación necesaria para llevar a cabo la optimización.

Si bien arquitecturas como las LSTM (Hochreiter y Schmidhuber, 1997) supusieron un avance muy significativo para la incorporación de dependencias de largo alcance, estas siguen presentando algunos problemas con las dependencias de muy largo alcance.

Figura 1. Diagrama básico representativo de una red recurrente con celdas LSTM



El despliegue de las redes que es necesario realizar durante el entrenamiento suele incrementar las probabilidades de explosión y desvanecimiento de gradientes, con los subsecuentes efectos sobre la optimización.

Cuando las secuencias de entrada son largas, es necesario desplegar la red en toda su extensión para calcular los gradientes asociados al tratamiento de la cadena. Esto incide no únicamente en el punto anterior (gradientes), sino también en las necesidades de recursos computacionales requeridos para su tratamiento (tiempo y espacio).

2. Arquitectura de los transformadores

En este apartado atenderemos a los detalles que conforman la arquitectura de los transformadores.

Los transformadores nacen con el objetivo de superar los desafíos propios de las redes recurrentes enunciados en el apartado anterior. Esta nueva arquitectura elimina la recurrencia. Todos los elementos de la secuencia se tratan al mismo tiempo. Los mecanismos de atención permiten la conexión directa de elementos situados a mucha distancia. Los problemas asociados a los gradientes también se minimizan, pues toda la secuencia se calcula al tiempo y, por tanto, los problemas de gradiente pasan a depender únicamente de la profundidad de la red. Por otro lado, sabemos que los problemas relacionados con la profundidad de la red quedaron resueltos años atrás con el advenimiento de las redes residuales (He y otros, 2016). Los transformadores son también redes residuales y, por tanto, resuelven también el problema de profundidad de la red utilizando los mismos mecanismos introducidos en He y otros (2016).

Finalmente, el entrenamiento en la práctica demuestra requerir menos épocas y, para cada una de ellas, el tratamiento en paralelo de la computación es mucho más sencillo de implementar.

2.1 El mecanismo de autoatención

El mecanismo de autoatención es la parte principal y distintiva de la arquitectura de los transformadores. Comprender su funcionamiento es clave para adentrarse en las especificidades de la arquitectura.

2.1.1 Definición formal

Sea $\mathbf{X}$ una secuencia de entrada formada por $\ell_{\mathcal{X}}$ elementos, cada uno de ellos de dimensión d_e , $\mathbf{X} \in \mathbb{R}^{d_e \times \ell_{\mathcal{X}}}$. Sea $\mathbf{Z}$ una secuencia denominada de contexto formada por $\ell_{\mathcal{Z}}$ elementos, cada uno de ellos de dimensión d_e , $\mathbf{Z} \in \mathbb{R}^{d_e \times \ell_{\mathcal{Z}}}$.

Sean $(\mathbf{W}_{\mathbf{q}} \in \mathbb{R}^{d_{attn} \times d_e}, \mathbf{b}_{\mathbf{q}} \in \mathbb{R}^{d_{attn}})$, $(\mathbf{W}_{\mathbf{k}} \in \mathbb{R}^{d_{attn} \times d_e}, \mathbf{b}_{\mathbf{k}} \in \mathbb{R}^{d_{attn}})$ y $(\mathbf{W}_{\mathbf{v}} \in \mathbb{R}^{d_{out} \times d_e}, \mathbf{b}_{\mathbf{v}} \in \mathbb{R}^{d_{out}})$ tres pares de matriz-vector representativos de tres transformaciones lineales aplicables a la secuencia de entrada $\mathbf{X}$ y a la secuencia de contexto $\mathbf{Z}$.

Sean $\mathbf{Q}$, $\mathbf{K}$ y $\mathbf{V}$ las matrices obtenidas después de aplicar las transformaciones lineales referidas, donde $\mathbf{Q} = \mathbf{W}_q \mathbf{X} + \mathbf{b}_q \mathbf{1}^T \in \mathbb{R}^{d_{attn} \times \ell_{\mathcal{X}}}$, $\mathbf{K} = \mathbf{W}_k \mathbf{Z} + \mathbf{b}_k \mathbf{1}^T \in \mathbb{R}^{d_{attn} \times \ell_{\mathcal{Z}}}$ y $\mathbf{V} = \mathbf{W}_v \mathbf{Z} + \mathbf{b}_v \mathbf{1}^T \in \mathbb{R}^{d_{out} \times \ell_{\mathcal{Z}}}$.

Sea $\mathbf{S}$ la función de **similitud**, definida en este caso como $\mathbf{S} = \frac{1}{\sqrt{d_{attn}}} \mathbf{K}^T \mathbf{Q} \in \mathbb{R}^{\ell_{\mathcal{Z}} \times \ell_{\mathcal{X}}}$.

Entonces, la ecuación de atención de la secuencia de entrada $\mathbf{X}$ sobre la secuencia de contexto $\mathbf{Z}$ se puede expresar como:

$$\tilde{\mathbf{V}} = \mathbf{V} \cdot \text{softmax}(\mathbf{S}) \quad \text{donde} \quad \tilde{\mathbf{V}} \in \mathbb{R}^{d_{out} \times \ell_{\mathcal{X}}} \quad (1)$$

La función *softmax* refiere a la aplicación de la función de normalización *softmax* por cada fila de la matriz. Por lo tanto, cada elemento de la matriz depende de todos los otros elementos pertenecientes a la misma fila.

Cuando la secuencia de entrada y el contexto coinciden, esto es $\mathbf{X} = \mathbf{Z}$, entonces hablamos de atención a la propia secuencia o, más brevemente, **autoatención**.

El algoritmo 1 muestra en versión algorítmica cómo se calcula la autoatención.

Algoritmo 1 Esquema básico de atención con una única consulta (versión secuencial)

Entrada: $e \in \mathbb{R}^{d_{in}}$, la representación *token* de entrada actual

Entrada: $e_t \in \mathbb{R}^{d_{in}}$, representación vectorial de los *tokens* de contexto $t \in [T]$

Salida: $\tilde{v} \in \mathbb{R}^{d_{out}}$, representación vectorial de la combinación de *token* y contexto.

Parámetros: $W_q, W_k \in \mathbb{R}^{d_{attn} \times d_{in}}, b_q, b_k \in \mathbb{R}^{d_{attn}}$, parámetros de proyección lineal de consulta y clave.

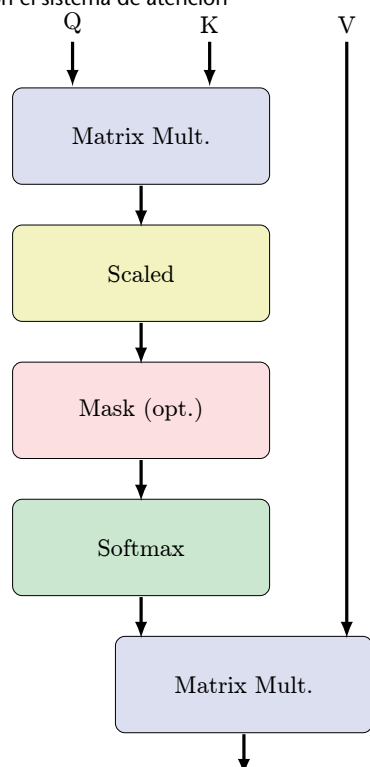
Parámetros: $W_v \in \mathbb{R}^{d_{out} \times d_{in}}, b_v \in \mathbb{R}^{d_{out}}$ parámetros de la proyección lineal de valor

- 1) $q \leftarrow W_q e + b_q$
 - 2) $\forall t : k_t \leftarrow W_k e_t + b_k$
 - 3) $\forall t : v_t \leftarrow W_v e_t + b_v$
 - 4) $\forall t : \alpha_t \leftarrow \frac{\exp(q^T k_t / \sqrt{d_{attn}})}{\sum_u \exp(q^T k_u / \sqrt{d_{attn}})}$
 - 5) **return** $\tilde{v} = \sum_{t=1}^T \alpha_t v_t$
-

Como se indica en la versión presentada en el algoritmo 1, este cálculo se realiza para todos los elementos de la secuencia. Con el objetivo de optimizar en tiempo la computación, es de interés paralelizar la computación en la

medida de lo posible. Una forma de hacerlo es mediante la configuración matricial del algoritmo. En la figura 2 se muestra mediante diagrama de bloques las operaciones matriciales necesarias para llevar a cabo el proceso. El módulo *Máscara*, que se muestra en el diagrama y se marca como opcional, es una operación que, de ser necesario, permite anular el efecto de parte de las secuencias. El algoritmo 2 es la versión matricial alternativa al algoritmo 1, que permite explotar las posibilidades de cálculo en paralelo y, por tanto, acelerar la computación.

Figura 2. Diagrama de bloques representativo de las operaciones matriciales involucradas en los cálculos relacionados con el sistema de atención



La operación de enmascarado permite el diseño de estrategias de entrenamiento alternativas para resolver problemas distintos utilizando la misma arquitectura. El sistema de **autoatención sin máscara** trata a toda la secuencia como contexto $Z = X$, ($Mask \equiv 1$), dando lugar a transformadores con contextos bidireccionales. Aplicando máscaras a los elementos de la secuencia situados a uno de los lados, tenemos los transformadores de contexto unidireccional, mientras que en aquellos en los que tenemos dos secuencias de entrada, es posible aplicar la secuencia cruzada enmascarando los elementos de una de las secuencias de entrada usando como contexto a la otra.

Algoritmo 2 $\hat{V} \leftarrow \text{atencion}(X, Z | W_{qkv}, \text{Mask})$ (versión matricial)

/* Calcula un (único) cabezal de atención con máscara */

Entrada: $X \in \mathbb{R}^{d_e \times \ell_x}$ y $Z \in \mathbb{R}^{d_e \times \ell_z}$, representaciones vectoriales de las secuencias principal y de contexto**Salida:** $\tilde{V} \in \mathbb{R}^{d_{out} \times \ell_x}$, representación vectorial actualizada de X , recogiendo la información de la secuencia Z **Parámetros:** W_{qkv} , consistente en: $W_q \in \mathbb{R}^{d_{attn} \times d_e}$, $b_q \in \mathbb{R}^{d_{attn}}$; $W_k \in \mathbb{R}^{d_{attn} \times d_e}$, $b_k \in \mathbb{R}^{d_{attn}}$; $W_v \in \mathbb{R}^{d_{out} \times d_e}$, $b_v \in \mathbb{R}^{d_{out}}$.**Hiperparámetros:** $\text{Mask} \in \{0, 1\}^{\ell_z \times \ell_x}$

- 1) $Q \leftarrow W_q X + b_q 1^T$ [[Query $\in \mathbb{R}^{d_{attn} \times \ell_x}$]]
- 2) $K \leftarrow W_k Z + b_k 1^T$ [[Key $\in \mathbb{R}^{d_{attn} \times \ell_z}$]]
- 3) $V \leftarrow W_v Z + b_v 1^T$ [[Value $\in \mathbb{R}^{d_{out} \times \ell_z}$]]
- 4) $S \leftarrow K^T Q$ [[Score $\in \mathbb{R}^{\ell_z \times \ell_x}$]]
- 5) $\forall t_z, t_x$, if $\neg \text{Mask}[t_z, t_x]$ then $S[t_z, t_x] \leftarrow -\infty$
- 6) **return** $\hat{V} = V \cdot \text{softmax}(S / \sqrt{d_{attn}})$

Informalmente, el transformador, a partir de una serie de atributos de representación de la secuencia de entrada, realiza una media ponderada entre los mismos basándose en un cálculo de similaridad. Los pesos aplicados a cada uno de estos atributos es proporcional al resultado de calcular la similaridad entre pares de atributos de entrada. Esta comparación se repite varias veces con los datos precedentes de la anterior capa. La primera capa efectúa comparaciones entre pares de atributos, la siguiente entre pares de pares de atributos, y así sucesivamente. Conforme se avanza en la profundidad de la red, el número de atributos combinados aumenta exponencialmente, permitiendo obtener en las últimas capas distintas formas de acción combinada de todos los atributos de la secuencia.

Autoatención

Query, *key* y *value* son las tres transformaciones lineales de los datos de entrada que constituyen la base del sistema de autoatención. El concepto de autoatención se deriva precisamente del hecho que las consultas Q se derivan de la naturaleza de la propia secuencia. El par Q - K permite establecer similitudes entre «lo buscado» (Q) y «lo encontrado» (K). El valor de salida será una combinación lineal de los valores (V) en base a la similitud entre cada par Q - K .

2.1.2 Definición conceptual: una analogía con las bases de datos

Los nombres originales utilizados para definir las transformaciones lineales usadas en el cálculo de la autoatención hacen referencia a una analogía conceptual con las consultas propias de las bases de datos relacionales.

En una base de datos tenemos consultas (*queries*, Q), claves (*keys*, K) y valores (*values*, V). Al realizar una consulta Q sobre un conjunto de claves $K_1, K_2, \dots, K_N$, la base de datos reporta como resultado una serie de valores $V_1, V_2, \dots, V_N$.

El mecanismo de atención de los transformadores viene a ser una versión probabilística de este proceso. Una función de similaridad compara la clave Q con

cada una de las claves K . El resultado es un vector que puede ser interpretado como la similaridad de la consulta Q con cada una de las claves K . Este valor sirve posteriormente para calcular un peso que se utiliza a continuación para calcular el valor final, como una combinación lineal de los valores de entrada. La función de similaridad puede ser definida de distintos modos (véase la ecuación 2). Es una decisión de diseño el escoger una u otra. Como hemos observado en las ecuaciones y algoritmos presentados en el subapartado anterior, los transformadores que estudiaremos utilizan como función de similaridad el producto escalar escalado.

$$s_i = f(Q, K_i) = \begin{cases} Q^T K_i & \text{producto escalar} \\ \frac{Q^T K_i}{d} & \text{producto escalar escalado} \\ Q^T W K_i & \text{producto escalar general} \\ W_Q^T + W_K^T K_i & \text{similaridad aditiva} \\ \text{otros} & \text{kernel, etc.} \end{cases} \quad (2)$$

Calculada la similaridad, obtenemos para una consulta concreta Q un valor por clave, es decir, un vector de dimensión igual al número de claves. Valores altos de similaridad indican un alto grado de coincidencia. Por tanto, como la intención es crear una función de búsqueda probabilística, se utiliza ese valor para calcular la probabilidad de coincidencia aplicando una función *softmax* (véase la ecuación 3), y obteniendo como resultado una probabilidad del valor de la posición i para una pareja consulta-clave determinada.

$$\omega_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)} \quad (3)$$

Finalmente, se obtendrá un conjunto de valores como combinación lineal de los valores fuente y el peso de cada uno de ellos en la comparación entre consulta-clave (véase la ecuación 4).

$$A = \sum \omega_i V_i \quad (4)$$

2.1.3 Autoatención múltiple

El mecanismo que hemos descrito en los subapartados anteriores permite realizar una transformación utilizando el superconjunto (Q, K, V) desde el espacio inicial $\mathbb{R}^{n \times d}$ para obtener como resultado un valor perteneciente al espacio $\mathbb{R}^{n \times d_v}$.

Una forma de ampliar las capacidades del sistema es aplicar varias transformaciones en paralelo del mismo tipo, pero con superconjuntos (Q, K, V) dis-

tintos. Cada uno de ellos reportando un valor distinto situado en el espacio $\mathbb{R}^{n \times d_v}$.

En la ecuación 5 se muestra la ecuación utilizada para calcular la atención múltiple.

$$\mathbf{M} = \text{Concat}_{i=1}^H [\mathbf{D}_i(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)] \mathbf{W}_O \quad (5)$$

Si el número de bloques de atención paralelos H se escoge de forma tal que $H = \frac{d}{d_v}$ entonces las dimensiones del espacio de salida serán iguales a las del espacio de entrada $\mathbb{R}^{n \times d}$. $\mathbf{W}_O$ es una matriz que realiza una transformación lineal posterior a la concatenación y permitiría eventualmente la modificación de las dimensiones del espacio de salida, en caso de que en el diseño pudiera ser de interés.

El mecanismo de atención múltiple permite aplicar en paralelo transformaciones diversas sobre los pares de atributos de entrada, aumentando la diversidad y complejidad de las comparaciones, así como mantener la dimensión del espacio de trabajo, hecho que puede ser de interés en arquitecturas que apilen bloques de la misma naturaleza.

El algoritmo 3 muestra la versión algorítmica del proceso.

Algoritmo 3 $\hat{\mathbf{V}} \leftarrow \text{atencion_multicabezal}(\mathbf{X}, \mathbf{Z} | \mathbf{W}_{qkv}, \text{Mask})$ (versión matricial)

/* Calcula la atención multicabezal (con máscara) */

Entrada: $\mathbf{X} \in \mathbb{R}^{d_x \times \ell_x}$ y $\mathbf{Z} \in \mathbb{R}^{d_z \times \ell_z}$, representaciones vectoriales de las secuencias principal y de contexto

Salida: $\tilde{\mathbf{V}} \in \mathbb{R}^{d_{out} \times \ell_x}$, representación vectorial actualizada de $\mathbf{X}$, recogiendo la información de la secuencia $\mathbf{Z}$

Parámetros: $\mathbf{W}_{qkv}$, consistente en: $\mathbf{W}_q \in \mathbb{R}^{d_{attn} \times d_x}$, $\mathbf{b}_q \in \mathbb{R}^{d_{attn}}$; $\mathbf{W}_k \in \mathbb{R}^{d_{attn} \times d_z}$, $\mathbf{b}_k \in \mathbb{R}^{d_{attn}}$; $\mathbf{W}_v \in \mathbb{R}^{d_{out} \times d_x}$, $\mathbf{b}_v \in \mathbb{R}^{d_{out}}$.

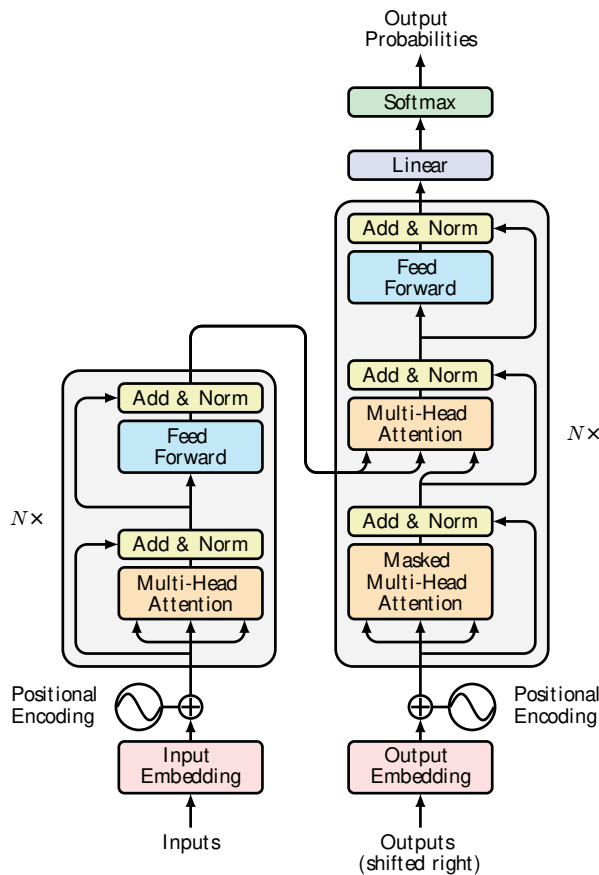
Hiperparámetros: $\text{Mask} \in \{0,1\}^{\ell_z \times \ell_x}$

- 1) Para $h \in [H]$:
 - 2) $\mathbf{Y}^h \leftarrow \text{atencion}(\mathbf{X}, \mathbf{Z} | \mathbf{W}_{qkv}^h, \text{Mask})$
 - 3) $\mathbf{Y}^h \leftarrow [\mathbf{Y}^1; \mathbf{Y}^2; \dots; \mathbf{Y}^H]$
 - 4) return $\hat{\mathbf{V}} = \mathbf{W}_O \mathbf{Y} + \mathbf{b}_O \mathbf{1}^T$
-

2.2 Transformadores

Llegados a este punto, estamos en condiciones de presentar la arquitectura del **transformador**. Como ya hemos indicado con anterioridad, este nuevo paradigma de diseño de red neuronal se introdujo con la publicación de Vaswani y otros (2017) y se planteó para su aplicación en sistemas de traducción automática.

Figura 3. Arquitectura del transformador introducida en «Attention is all you need» (Vaswani y otros, 2017)



En la figura 3 se muestra el diagrama de los bloques integrantes de la red de transformadores tal como fue publicado por primera vez en Vaswani y otros (2017). Cada elemento de la secuencia de entrada se convierte en una representación numérica (*input embedding*) para posteriormente combinarse (suma en este caso) con una función que codifica la posición del elemento en la secuencia (*positional encoding*). Este valor sirve de entrada a una pila de N bloques denominados *codificadores*. Los atributos fruto de la codificación serán una de las entradas que alimentarán a la red de **decodificadores**. El objetivo de esta red es predecir la siguiente palabra y, para ello, utiliza no únicamente la información procedente de la cadena de la entrada a través del codificador, sino también todos los elementos de la cadena de salida producidos hasta el momento.

En los subapartados siguientes vamos a describir con detalle cada una de las partes integrantes de la arquitectura, agrupándolas en sus elementos constitutivos de mayor nivel.

2.2.1 Notación

Sea V un conjunto finito denominado **vocabulario**, denotado como $[N_V] := \{1, \dots, N_V\}$. Puede estar formado por letras o palabras completas, aunque típicamente está formado por trozos de palabras denominados *tokens*.

Sea $\mathbf{x} \equiv x[1 : \ell] \equiv x[1]x[2]...x[\ell] \in V^*$ una secuencia de *tokens*, por ejemplo, una frase, un párrafo o un documento.

Siguiendo la notación matemática y en contra de lo establecido en algunos lenguajes de programación como C o Python, el primer índice de matrices y vectores utilizados en este módulo será el uno. $X[1 : \ell]$, por ejemplo, refiere a la cadena de caracteres que va desde el primero hasta el elemento ℓ , ambos incluidos.

Para una matriz $M \in \mathbb{R}^{d \times d'}$, escribimos $M[i, :] \in \mathbb{R}^{d'}$ para referirnos a la fila i y $M[:, j] \in \mathbb{R}^d$ para la columna j .

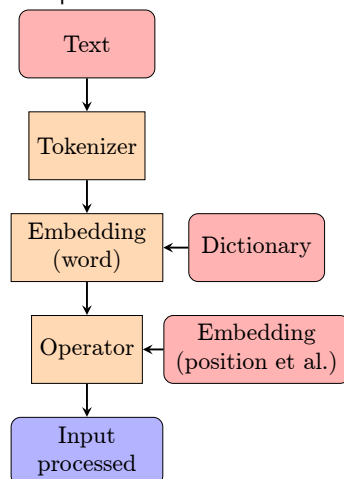
Usamos la convención matriz $\times$ columna más común en matemáticas en vez de fila $\times$ matriz más típica de la mayoría de la literatura de transformadores, esto es, las matrices están traspuestas.

2.2.2 Elementos constitutivos del transformador

Tratamiento de entrada

En la figura 4 se muestra el proceso de tratamiento de entrada propio del transformador. El texto se alimenta a un *tokenizador*, cuya función es particionar la secuencia en sus elementos constituyentes. Cada elemento es representado por un vector ortogonal que tiene un 1 en la posición del diccionario identificativa del elemento y ceros en el resto. A continuación se realiza una transformación lineal para comprimir la representación inicial en un vector denso de dimensión menor. Finalmente, mediante un operador, que suele ser la suma (pero nada impide que pudiera ser por ejemplo la concatenación u otro distinto), se añade la información referente a la posición que ocupa el elemento dentro de la secuencia. La representación numérica de la secuencia ya está lista para ser alimentada a los bloques de procesamiento siguientes.

Figura 4. Esquema típico de las operaciones necesarias para convertir una entrada de texto a formato necesario para ser tratado por el transformador



La capa de entrada se encarga del preprocesamiento de los datos para adecuarlos a un formato que sea interpretable por el transformador. Los transformadores se diseñaron inicialmente como forma alternativa de tratamiento de los datos secuenciales. El primer paso para el tratamiento de este tipo de datos es la partición de la secuencia en sus elementos constitutivos. Este proceso es conocido habitualmente como *tokenización*. Esta partición se puede hacer de muchos modos: a nivel de palabra, partes de palabra o bien caracteres.

En los diseños actuales de transformadores, lo más habitual es utilizar particiones a nivel de partes de palabra. Muchas palabras están constituidas por distintos monemas (unidades mínimas de significado). Particionar a nivel de palabra tiene el inconveniente de que muchas de las unidades elementales de significado no se separan. Esto suele ser un inconveniente que dificulta el funcionamiento de modelos interpretativos del lenguaje.

De entrada se podría conjeturar que una separación a nivel de carácter permitiría al modelo reconstruir las unidades mínimas de significado. Los artículos publicados indican explícitamente, que los resultados prácticos, al menos hasta el momento, indican que la partición a nivel de carácter es demasiado agresiva, con resultados significativamente inferiores a los obtenidos utilizando particiones a nivel de partes de palabra.

Entre los particionadores más típicos utilizados con transformadores tenemos *BPE (Byte Pair Encoding)* (Gage, 1994), *Unigram* (Kudo, 2018) y *WordPiece* (Devlin y otros, 2018). La diferencia existente entre estos se basa en la estrategia seguida para escoger los pares de caracteres a concatenar para elegir cada una de las subpalabras para formar el diccionario.

Cada modelo preentrenado está diseñado para ser usado con un diccionario específico derivado del particionador escogido.

Para el caso de los transformadores para el tratamiento de imagen, la propuesta existente hasta el momento consiste en particionar la imagen en trozos no superpuestos de dimensiones reducidas y alimentarlos secuencialmente.

Si bien la preparación de las entradas descritas son los modos actuales establecidos de hacerlo, no existe una razón para que esto no pueda ser de otro modo. Por lo tanto, es posible que en el futuro puedan surgir maneras alternativas de codificar las entradas.

Algoritmo 4 *Embedding* de un *token*

Entrada: $v \in V \cong [N_V]$, identificador del *token* (p.e. one-hot)

Salida: $e \in \mathbb{R}^{d_e}$, la representación vectorial de un *token*

Parámetros: $W_e \in \mathbb{R}^{d_e \times N_V}$, la matriz de *embedding*

retorna: $e = W_e[:, v]$

Algoritmo 5 *Embedding* de la posición

Entrada: $\ell \in \ell_{max}$, posición del *token* dentro de la secuencia

Salida: $\mathbf{e}_p \in \mathbb{R}^{d_e}$, la representación vectorial de la posición

Parámetros: $\mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{max}}$, la matriz de *embeddings* de posición

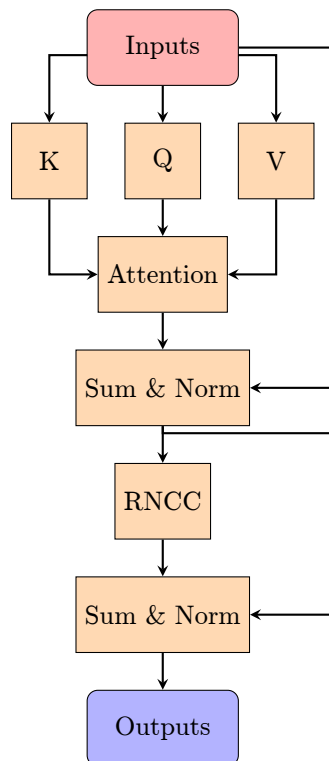
retorna: $\mathbf{e}_p = \mathbf{W}_p[:, \ell]$

Codificadores

El codificador está integrado por un cabezal de autoatención múltiple seguido de una red completamente conectada. Ambas capas forman parte de una red residual y, por lo tanto, integran a continuación de cada una de esas dos capas la suma de la entrada y su normalización.

En la figura 5 se muestra el diagrama de bloques de los elementos integrantes de un codificador.

Figura 5. Esquema típico de los elementos integrantes de un codificador. La red de codificación integra varios de estos elementos en serie. La salida de uno sirve de entrada para el siguiente.

**Decodificadores**

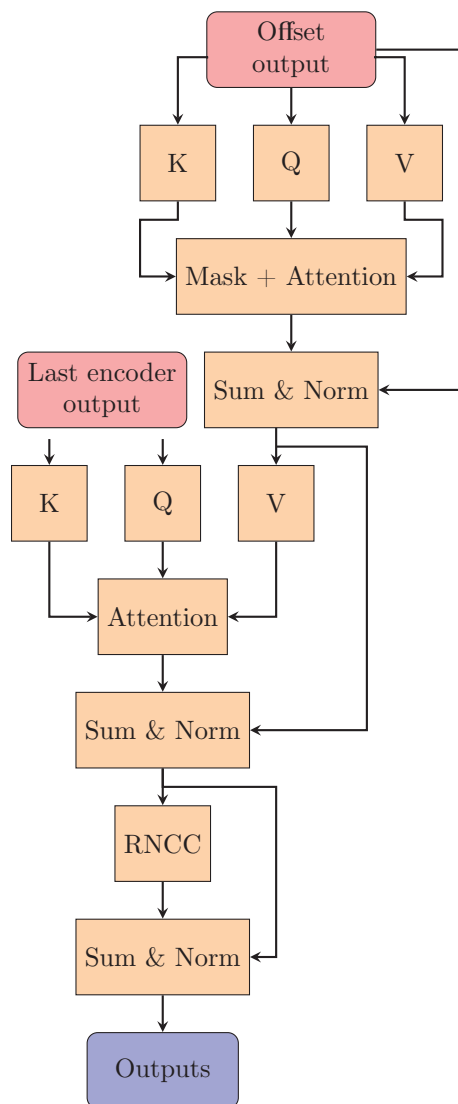
El decodificador es un bloque integrado por un cabezal de autoatención múltiple que recibe como entradas los elementos de la cadena de salida de la red del transformador desplazados una posición. Para asegurar que la predicción

se realiza usando únicamente la información precedente, pero no la posterior, integra una matriz de enmascarado que permite anular el efecto de los elementos posteriores de la cadena de salida.

A continuación de este primer bloque, existe un segundo que recibe como entrada la salida del primero, así como la salida procedente de la codificación de la cadena de entrada. La salida de este bloque se alimenta a una capa completamente conectada. Todas estas etapas son también capas residuales y están seguidas de su correspondiente suma con la entrada y normalización. La red de decodificación apila en serie N elementos de este tipo.

En la figura 6 se muestra el diagrama de bloques de los elementos integrantes de un decodificador.

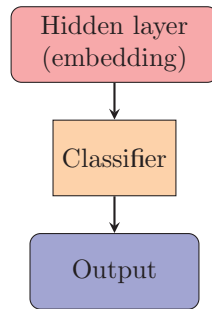
Figura 6. Esquema típico de los elementos integrantes de un decodificador. La red de decodificación integra varios de estos elementos en serie. La salida de uno sirve de entrada para el siguiente. A todos ellos se alimenta la salida de la red de codificadores.



Tratamiento de salida

En la capa de salida es necesario realizar una conversión desde el espacio de representación interna hasta el de representación externa (véase la figura 7). Es la transformación inversa a la operación de *embedding*, donde pasamos del vector representativo del diccionario a una representación interna.

Figura 7. Esquema típico de las operaciones necesarias para convertir la última capa oculta del transformador en una salida



Existen dos formas de llevar a cabo la conversión: la primera es tratar a la capa como independiente, realizando el aprendizaje de los pesos de conversión, mientras que la segunda es utilizar la matriz inversa del *embedding*, sin realizar un aprendizaje independiente.

En Vaswani y otros (2017) se utiliza la estrategia definida en Press y Wolf (2016) de ligar la transformación de la capa de entrada a la capa de salida. Uno de los pasos iniciales para preparar las entradas pasa por convertir la representación *one-hot* de su posición en el diccionario por una representación «densa» de las dimensiones del modelo. Este paso se hace mediante una transformación lineal que reduce las dimensiones desde las del diccionario a la interna de trabajo del modelo. La idea presentada en Press y Wolf (2016) y utilizada posteriormente en Vaswani y otros (2017) pasa por utilizar la misma matriz de transformación tanto para las entradas como para las salidas. Siendo esto así, el clasificador de salida pasa a ser una transformación lineal con los mismos pesos que la de la entrada.

El algoritmo 6 muestra la versión algorítmica del proceso.

Algoritmo 6 *Embedding* inverso

/* Convierte la representación interna de la red en un elemento */

Entrada: $\mathbf{e} \in \mathbb{R}^{d_e}$, representación interna de un elemento

Salida: $\mathbf{p} \in \Delta(V)$, vector que contiene la representación distribución de probabilidad sobre el conjunto de elementos integrantes del vocabulario

Parámetros: $\mathbf{W}_u \in \mathbb{R}^{N_v \times d_e}$, matriz de *embedding* inversa

return $\mathbf{p} = \text{softmax}(\mathbf{W}_u \mathbf{e})$

Normalización de capas

Las capas de normalización están presentes como bloque constituyente tanto en los codificadores como en los decodificadores. Su finalidad es el control explícito de la media y la varianza de la activación de cada neurona.

El algoritmo 7 muestra de forma detallada el proceso que tiene lugar en dichas capas.

Algoritmo 7 $\hat{\mathbf{e}} \leftarrow \text{normalizacion_de_capa}(\mathbf{e}|\boldsymbol{\gamma},\boldsymbol{\beta})$

/ Normaliza las activaciones $\mathbf{e}$ de la capa */*

Entrada: $\mathbf{e} \in \mathbb{R}^{d_e}$, activaciones de entrada sin normalizar

Salida: $\hat{\mathbf{e}} \in \mathbb{R}^{d_e}$, activaciones normalizadas

Parámetros: $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_e}$, vector de escalado y desplazamiento

- 1) $\mathbf{m} \leftarrow \sum_{i=1}^{d_e} \mathbf{e}[i]/d_e$
 - 2) $\nu \leftarrow \sum_{i=1}^{d_e} (\mathbf{e}[i] - \mathbf{m})^2/d_e$
 - 3) **return** $\hat{\mathbf{e}} = \frac{\mathbf{e} - \mathbf{m}}{\sqrt{\nu}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}$, donde $\odot$ denota multiplicación componente-componente
-

2.3 Tipos de transformadores

En el subapartado anterior se ha introducido la arquitectura del transformador mostrando el diagrama original introducido en Vaswani y otros (2017). Como ya hemos indicado con anterioridad, la propuesta inicial se hizo como medio alternativo de resolución de problemas de traducción de secuencia a secuencia y representa el primero de los tipos de modelos posibles de transformadores. En este subapartado presentaremos modificaciones de la arquitectura original que surgieron posteriormente y que han dado lugar a subfamilias de transformadores especializadas en la resolución de problemas distintos. Actualmente podemos distinguir entre tres tipos de modelos o arquitecturas: los autorregresivos, los de autocodificación y los de conversión de secuencia a secuencia.

2.3.1 Transformadores secuencia a secuencia

Los modelos de secuencia a secuencia utilizan dos unidades básicas de diseño denominadas codificadores y decodificadores. Su finalidad es la conversión de una secuencia de entrada en otra distinta. Sus aplicaciones principales están relacionadas con la traducción, el resumen y la respuesta a preguntas. El modelo original es un ejemplo de este tipo de aplicaciones. T5 (Raffel y otros, 2020) sería un ejemplo presentado después del artículo general de la misma arquitectura usada para resolver otro tipo de tareas más específicas.

El algoritmo 8 recoge los detalles de implementación de lo que sería un transformador de secuencia a secuencia.

Algoritmo 8 $P \leftarrow \text{transformadorCD}(\mathbf{x} \mid \boldsymbol{\theta})$

/* Transformador codificador/decodificador o de secuencia a secuencia */

Entrada: $\mathbf{z}, \mathbf{x} \in V^*$, dos secuencias de identificadores de *tokens***Salida:** $\mathbf{P} \in (0,1)^{N_V \times \text{longitud}(\mathbf{x})}$, donde la columna t de $\mathbf{P}$ representa $\hat{P}_{\boldsymbol{\theta}}(\mathbf{x}[t+1] \mid \mathbf{x}[1:t], \mathbf{z})$ **Hiperparámetros:** $\ell_{\max}, L_{\text{enc}}, L_{\text{dec}}, H, d_e, d_{\text{mlp}}, d_f \in \mathbb{N}$ **Parámetros:** $\boldsymbol{\theta}$ que incluye a: $\mathbf{W}_e \in \mathbb{R}^{d_e \times N_V}, \mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{\max}}$, matrices de *embedding* de *token* y posiciónPara $l \in [L_{\text{enc}}]$: $\mathbf{W}_{\text{qkv},l}^{\text{enc}}$, autoatención de la capa l $\gamma_l^1, \beta_l^1, \gamma_l^2, \beta_l^2 \in \mathbb{R}^{d_e}$, dos conjuntos de normalización de capas $\mathbf{W}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}}}, \mathbf{W}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}}}$ Para $l \in [L_{\text{dec}}]$: $\mathbf{W}_{\text{qkv},l}^{\text{dec}}, \mathbf{W}_{\text{qkv},l}^{\text{ed}}$, autoatención y autoatención cruzada de la capa l $\gamma_l^3, \beta_l^3, \gamma_l^4, \beta_l^4, \gamma_l^5, \beta_l^5 \in \mathbb{R}^{d_e}$, tres conjuntos de normalización de capas $\mathbf{W}_{\text{mlp}3}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}3}^l \in \mathbb{R}^{d_{\text{mlp}}}, \mathbf{W}_{\text{mlp}4}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}4}^l \in \mathbb{R}^{d_{\text{mlp}}}$ $\mathbf{W}_u \in \mathbb{R}^{N_V \times d_e}$, matriz inversa de *embedding*

/* Codificación de la secuencia de contexto */

```

1   $\ell_Z \leftarrow \text{longitud}(\mathbf{z})$ 
2  para  $t \in [\ell_Z] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, \mathbf{z}[t]] + \mathbf{W}_p[:, t]$ 
3   $\mathbf{Z} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell]$ 
4  para  $\ell = 1, 2, \dots, L_{\text{enc}}$  hacer
5       $\mathbf{Z} \leftarrow \mathbf{Z} + \text{atencion\_multicabezal}(\mathbf{Z} \mid \mathbf{W}_{\text{qkv},\ell}^{\text{enc}}, \text{Mask} \equiv 1)$ 
6      para  $t \in [\ell_Z] : \mathbf{Z}[:, t] \leftarrow \text{normalizacion}(\mathbf{Z}[:, t] \mid \gamma_\ell^1, \beta_\ell^1)$ 
7       $\mathbf{Z} \leftarrow \mathbf{Z} + \mathbf{W}_{\text{mlp}2}^l \text{ReLU}(\mathbf{W}_{\text{mlp}1}^l \mathbf{Z} + \mathbf{b}_{\text{mlp}1}^l \mathbf{1}^T) + \mathbf{b}_{\text{mlp}2}^l \mathbf{1}^T$ 
8      para  $t \in [\ell_Z] : \mathbf{Z}[:, t] \leftarrow \text{normalizacion}(\mathbf{Z}[:, t] \mid \gamma_\ell^2, \beta_\ell^2)$ 
9  fpara

```

/* Decodificación de la primera secuencia condicionada por el contexto */

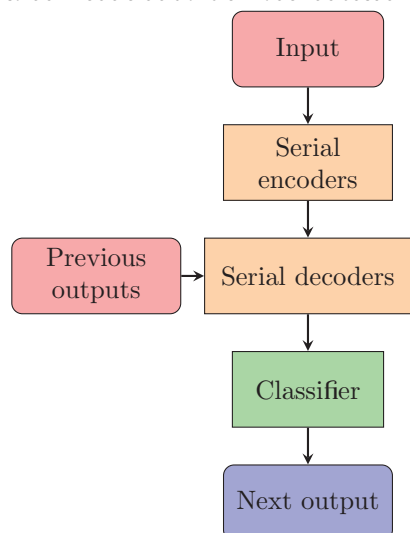
```

10  $\ell_X \leftarrow \text{longitud}(\mathbf{x})$ 
11 para  $t \in [\ell_X] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, \mathbf{x}[t]] + \mathbf{W}_p[:, t]$ 
12  $\mathbf{X} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell]$ 
13 para  $i = 1, 2, \dots, L_{\text{dec}}$  hacer
14      $\mathbf{X} \leftarrow \mathbf{X} + \text{atencion\_multicabezal}(\mathbf{X} \mid \mathbf{W}_{\text{qkv},i}^{\text{dec}}, \text{Mask}[t, t'] = [[t \leq t']])$ 
15     para  $t \in [\ell_X] : \mathbf{X}[:, t] \leftarrow \text{normalizacion}(\mathbf{X}[:, t] \mid \gamma_i^3, \beta_i^3)$ 
16      $\mathbf{X} \leftarrow \mathbf{X} + \text{atencion\_multicabezal}(\mathbf{X}, \mathbf{Z} \mid \mathbf{W}_{\text{qkv},i}^{\text{ed}}, \text{Mask} \equiv 1)$ 
17     para  $t \in [\ell_X] : \mathbf{X}[:, t] \leftarrow \text{normalizacion}(\mathbf{X}[:, t] \mid \gamma_i^4, \beta_i^4)$ 
18      $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{W}_{\text{mlp}4}^l \text{ReLU}(\mathbf{W}_{\text{mlp}3}^l \mathbf{X} + \mathbf{b}_{\text{mlp}3}^l \mathbf{1}^T) + \mathbf{b}_{\text{mlp}4}^l \mathbf{1}^T$ 
19 fpara
20 return  $P = \text{softmax}(\mathbf{W}_u \mathbf{X})$ 

```

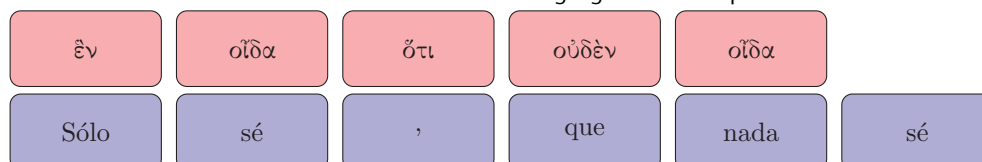

En la figura 8 se representa el diagrama funcional típico representativo de esta arquitectura. Este diagrama es equivalente al de la figura 3, mostrando los elementos constitutivos al más alto nivel, es decir: los bloques de tratamiento de la secuencia de entrada, los codificadores, decodificadores y finalmente la capa de clasificación y salida.

Figura 8. Arquitectura típica del modelo de transformador de secuencia a secuencia



En la figura 9 mostramos un ejemplo de una secuencia típica de entrada y otra de salida. Las secuencias mostradas serían un ejemplo de una traducción de una secuencia inicial en griego clásico a otra secuencia en español. Al modelo se le entraría la primera y se obtendría como salida la segunda.

Figura 9. Ejemplo de un conjunto de entrada y salida común de un modelo secuencia a secuencia. En este caso sería un modelo de traducción de griego clásico al español.



En el algoritmo 9 se muestra lo que sería el código necesario para realizar la optimización del modelo utilizando un conjunto de datos de entrada.

Algoritmo 9 $\hat{\theta} \leftarrow \text{entrenamientoCD}(\mathbf{z}_{1:N_{data}}, \mathbf{x}_{1:N_{data}}, \theta)$

/* Entrenamiento de un transformador de secuencia a secuencia */

Entrada: $(\mathbf{z}_n, \mathbf{x}_n)_{n=1}^{N_{data}}$, un conjunto de datos de parejas de secuencias**Entrada:** θ , valores de partida de los parámetros del transformador**Salida:** $\hat{\theta}$, valor de los parámetros después del entrenamiento**Hiperparámetros:** $N_{epochs} \in \mathbb{N}, \eta \in (0, \infty)$

```

1  para  $i = 1, 2, \dots, N_{epochs}$  hacer
2      para  $n = 1, 2, \dots, N_{data}$  hacer
3           $\ell \leftarrow \text{longitud}(\mathbf{x}_n)$ 
4           $\mathbf{P}_\theta \leftarrow \text{transformador\_de\_codificadores\_decodificadores}(\mathbf{z}_n, \mathbf{x}_n \mid \theta)$ 
5           $\text{loss}(\theta) = -\sum_{t=1}^{\ell-1} \log P(\theta)[x_n[t+1], t]$ 
6           $\theta \leftarrow \theta - \eta \nabla \text{loss}(\theta)$ 
7      fpara
8  fpara
9  retorna:  $\hat{\theta} = \theta$ 

```

2.3.2 Transformadores bidireccionales o de autocodificación

Los modelos de autocodificación son aquellos que están preentrenados para la corrección de entradas corrompidas artificialmente. Esto se hace para entrenar al modelo en la reconstrucción del texto original. Este tipo de modelos se corresponde con la parte del codificador de la arquitectura original en el sentido que no utilizan máscaras para anular el efecto del texto posterior. Estos modelos construyen una representación bidireccional de la frase en su totalidad y pueden ser posteriormente entrenados para la resolución de tareas específicas como la generación de texto, aunque su aplicación principal es la clasificación de frases. Un ejemplo típico de estos modelos sería **BERT** (Devlin y otros, 2018).

El algoritmo 10 recoge los detalles de implementación de lo que sería un transformador bidireccional, esto es, un transformador formado únicamente por codificadores.

En el algoritmo 11 se muestra lo que sería el código necesario para realizar la optimización del modelo utilizando un conjunto de datos de entrada.

Algoritmo 10 $P \leftarrow \text{codificador}(x \mid \theta)$

/* Transformador formado por codificadores, bidireccional o autocodificador */

Entrada: $x \in V^*$, una secuencia de identificadores de *tokens*

Salida: $P \in (0,1)^{N_V \times \text{longitud}(x)}$, donde cada columna de P es una distribución sobre todo el vocabulario

Hiperparámetros: $\ell_{\max}, L, H, d_e, d_{mlp}, d_f \in \mathbb{N}$

Parámetros: θ que incluye a:

$W_e \in \mathbb{R}^{d_e \times N_V}, W_p \in \mathbb{R}^{d_e \times \ell_{\max}}$, matrices de *embedding* de *token* y posición

Para $l \in [L]$:

$W_{qkv,l}$, parámetros de autoatención de la capa l

$\gamma_l^1, \beta_l^1, \gamma_l^2, \beta_l^2 \in \mathbb{R}^{d_e}$, dos conjuntos de normalización de capas

$W_{mlp1}^l \in \mathbb{R}^{d_{mlp} \times d_e}, b_{mlp1}^l \in \mathbb{R}^{d_{mlp}}, W_{mlp2}^l \in \mathbb{R}^{d_{mlp} \times d_e}, b_{mlp2}^l \in \mathbb{R}^{d_{mlp}}$

$W_f \in \mathbb{R}^{d_f \times d_e}, b_f \in \mathbb{R}^{d_f}, \gamma, \beta \in \mathbb{R}^{d_f}$, proyección lineal y normalización final

$W_u \in \mathbb{R}^{N_V \times d_e}$, matriz inversa de *embedding*

- 1 $\ell \leftarrow \text{longitud}(x)$
- 2 para $t \in [\ell] : e_t \leftarrow W_e[:, x[t]] + W_p[:, t]$
- 3 $X \leftarrow [e_1, e_2, \dots, e_\ell]$
- 4 para $\ell = 1, 2, \dots, L$ hacer
 - 5 $X \leftarrow X + \text{atencion_multicabezal}(X \mid W_{qkv,l}, \text{Mask} \equiv 1)$
 - 6 para $t \in [\ell] : X[:, t] \leftarrow \text{normalizacion}(X[:, t] \mid \gamma_l^1, \beta_l^1)$
 - 7 $X \leftarrow X + W_{mlp2}^l \text{GELU}(W_{mlp1}^l X + b_{mlp1}^l \mathbf{1}^T) + b_{mlp2}^l \mathbf{1}^T$
 - 8 para $t \in [\ell] : X[:, t] \leftarrow \text{normalizacion}(X[:, t] \mid \gamma_l^2, \beta_l^2)$
- 9 **fpara**
 - 10 $X \leftarrow \text{GELU}(W_f X + b_f \mathbf{1}^T)$
 - 11 para $t \in [\ell] : X[:, t] \leftarrow \text{normalizacion}(X[:, t] \mid \gamma, \beta)$
- 12 **return** $P = \text{softmax}(W_u X)$

Algoritmo 11 $\hat{\theta} \leftarrow \text{entrenamientoC}(\mathbf{x}_{1:N_{data}}, \theta)$

/* Entrenamiento mediante modelización mediante máscara */

Entrada: $\{\mathbf{x}_n\}_{n=1}^{N_{data}}$, un conjunto de datos de secuencias**Entrada:** θ , valores de partida de los parámetros del transformador**Salida:** $\hat{\theta}$, valor de los parámetros después del entrenamiento**Hiperparámetros:** $N_{epochs} \in \mathbb{N}, \eta \in (0, \infty), p_{mask} \in (0, 1)$

```

1  para  $i = 1, 2, \dots, N_{epochs}$  hacer
2      para  $n = 1, 2, \dots, N_{data}$  hacer
3           $\ell \leftarrow \text{longitud}(\mathbf{x}_n)$ 
4          para  $t = 1, 2, \dots, \ell$  hacer
5               $\tilde{x}_n[t] \leftarrow \text{enmascara\_token}$  o  $x_n[t]$  aleatoriamente con
                  probabilidad  $p_{mask}$  o  $1 - p_{mask}$ 
6          fpara
7               $\tilde{T} \leftarrow \{t \in [\ell] : \tilde{x}_n[t] = \text{enmascara\_token}\}$ 
8               $\mathbf{P}(\theta) \leftarrow \text{transformador\_de\_codificadores}(\tilde{\mathbf{x}}_n \mid \theta)$ 
10              $\text{loss}(\theta) = -\sum_{t \in \tilde{T}} \log P(\theta)[x_n[t], t]$ 
11              $\theta \leftarrow \theta - \eta \nabla \text{loss}(\theta)$ 
12         fpara
13     fpara
14     retorna:  $\hat{\theta} = \theta$ 

```

La red de codificación apila en serie L elementos del mismo tipo. La salida de cada bloque sirve de entrada para el siguiente. Las dimensiones de la transformación se mantienen, por lo que entrada y salida tienen el mismo tamaño.

Figura 10. Arquitectura típica del modelo de transformador autocodificador

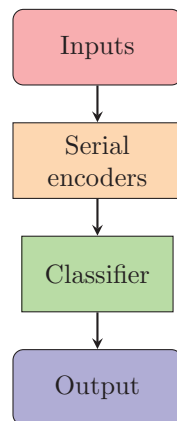


Figura 11. Ejemplo de entradas y salidas típicas de los modelos de autocodificación. En este caso se presentan tres entradas posibles con sus valores de salida esperados.

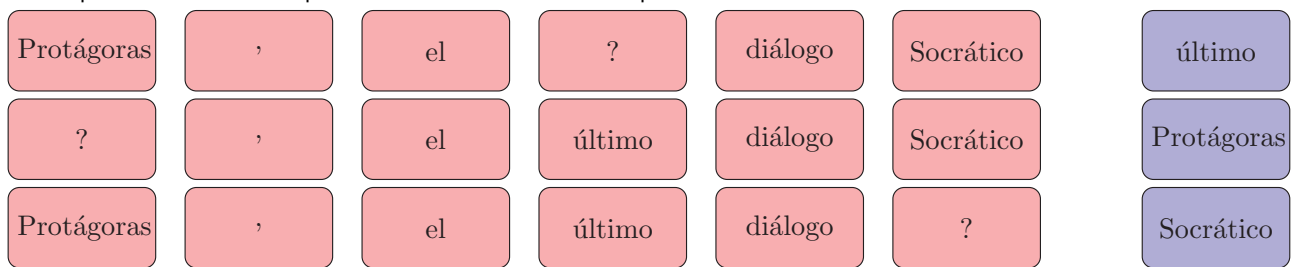


Tabla 1. Características de algunos de los primeros modelos bidireccionales más conocidos

	BERT-b	BERT-I	XLNet-I	RoBERTa-I	DistilBERT-b	ALBERT-I
Arquitectura						
n_{param}	108×10^6	340×10^6	340×10^6	355×10^6	66×10^6	18×10^6
T_{param}	432 MB	1360 MB	1360 MB	1420MB	264MB	72 MB
ℓ_{max}	512	512				
L_{enc}	12	24	24	24	6	24
H	12	16	16	16	12	16
d_{attn}	64	64	64	64		8
d_e	768	1024	1024	1024		128
d_{mlp}	3072	4096		4096		512
N_V	30522					
$c_{entrada}$	WordPiece					
Entrenamiento						
T_{datos}	16 GB	16 GB	113 GB	160 GB	16 GB	16 GB
n_{tokens}	3.3×10^9	3.3×10^9	33×10^9		3.3×10^9	
$ratio_{compr}$	30.5	9.7	97.0		50.0	
Optimizador				Adam		
BS				8000		
Rendimiento						
GLUE		base	+15%	+20%	−3%	+25%

2.3.3 Transformadores autorregresivos

Los modelos autorregresivos son aquellos que a partir de una secuencia de entrada inicial generan nuevas salidas que a su vez se incorporan a la secuencia inicial como entrada para generar nuevas salidas. Su principal aplicación es la generación de texto, aunque es posible ajustarlos con un entrenamiento posterior para adaptarlos a la resolución de problemas específicos. Un ejemplo de este tipo de modelos sería la familia de **GPT**.

El algoritmo 12 recoge los detalles de implementación de lo que sería un transformador autorregresivo, es decir, un transformador formado únicamente por decodificadores.

N_V

El tamaño del vocabulario depende de las características del modelo. Este valor variará dependiendo de si se ha entrenado para ser sensible mayúsculas, para un único idioma o bien para distintos lenguajes. Es un valor orientativo correspondiente a un modelo preentrenado sin especificar, con objetivo únicamente orientativo.

Algoritmo 12 $P \leftarrow \text{transformadorD}(\mathbf{x} \mid \boldsymbol{\theta})$

/* Transformador formado por decodificadores o autorregresivo */

Entrada: $\mathbf{x} \in V^*$, una secuencia de identificadores de *tokens***Salida:** $\mathbf{P} \in (0,1)^{N_V \times \text{longitud}(\mathbf{x})}$, donde la columna t de $\mathbf{P}$ representa $\hat{P}_{\boldsymbol{\theta}}(\mathbf{x}[t+1] \mid \mathbf{x}[1:t])$ **Hiperparámetros:** $\ell_{\max}, L, H, d_e, d_{\text{mlp}}, d_f \in \mathbb{N}$ **Parámetros:** $\boldsymbol{\theta}$ que incluye a: $\mathbf{W}_e \in \mathbb{R}^{d_e \times N_V}, \mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{\max}}$, matrices de *embedding* de *token* y posiciónPara $l \in [L]$: $\mathbf{W}_{qkv,l}$, parámetros de autoatención de la capa l $\gamma_l^1, \beta_l^1, \gamma_l^2, \beta_l^2 \in \mathbb{R}^{d_e}$, dos conjuntos de normalización de capas $\mathbf{W}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}}}, \mathbf{W}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}}}$ $\mathbf{W}_f \in \mathbb{R}^{d_f \times d_e}, \mathbf{b}_f \in \mathbb{R}^{d_f}, \gamma, \beta \in \mathbb{R}^{d_f}$, proyección lineal y normalización final $\mathbf{W}_u \in \mathbb{R}^{N_V \times d_e}$, matriz inversa de *embedding*

- 1 $\ell \leftarrow \text{longitud}(\mathbf{x})$
- 2 para $t \in [\ell] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, \mathbf{x}[t]] + \mathbf{W}_p[:, t]$
- 3 $\mathbf{X} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell]$
- 4 para $\ell = 1, 2, \dots, L$ hacer
 - 5 para $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalizacion}(\mathbf{X}[:, t] \mid \gamma_l^1, \beta_l^1)$
 - 6 $\mathbf{X} \leftarrow \mathbf{X} + \text{atencion_multicabezal}(\mathbf{X} \mid \mathbf{W}_{qkv,l}, \text{Mask}[t, t'] = [[t \leq t']])$
 - 7 para $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalizacion}(\mathbf{X}[:, t] \mid \gamma_l^2, \beta_l^2)$
 - 8 $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{W}_{\text{mlp}2}^l \text{GELU}(\mathbf{W}_{\text{mlp}1}^l \mathbf{X} + \mathbf{b}_{\text{mlp}1}^l \mathbf{1}^T) + \mathbf{b}_{\text{mlp}2}^l \mathbf{1}^T$
 - 9 **fpara**
 - 10 para $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalizacion}(\mathbf{X}[:, t] \mid \gamma, \beta)$
 - 11 **return** $P = \text{softmax}(\mathbf{W}_u \mathbf{X})$

Igual que en los codificadores, la red de decodificación apila en serie N elementos del mismo tipo. La salida de cada bloque sirve de entrada para el siguiente. Las dimensiones de la transformación se mantienen, por lo que entrada y salida tienen el mismo tamaño.

Figura 12. Arquitectura de transformador autorregresivo

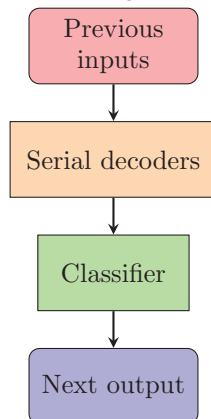


Figura 13. Ejemplo de entradas y salidas típicas de los modelos autorregresivos. En este caso se presentan tres entradas posibles con sus respectivas salidas esperadas.

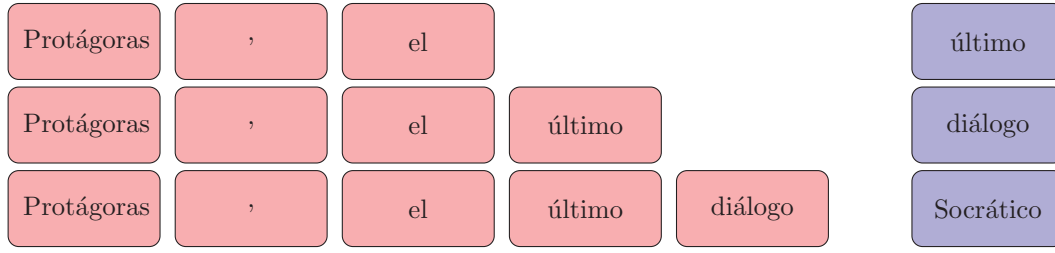


Tabla 2. Características de los modelos autorregresivos de la familia GPT.

	GPT-1	GPT-2	GPT-3
Arquitectura			
n_{param}	117×10^6	1.5×10^9	175×10^9
T_{param}	468 MB	6 GB	700GB
ℓ_{max}	512	1024	2048
L_{dec}	12	48	96
H	12	12	96
d_{attn}	64		128
d_{mlp}	3072		
N_V	40478	50257	50257
$c_{entrada}$	BPE	BPE	BPE
d_e	768	1600	12288
Entrenamiento			
T_{datos}	1 GB	40 GB	2 TB
n_{tokens}	25×10^7	10×10^9	499×10^9
$ratio_{compr}$	2.13	6.66	2.85
Optimizador	Adam	Adam	Adam
BS	64	512	3.2×10^6

En el algoritmo 13 se muestra el conjunto de operaciones que se llevan a cabo en un bucle de entrenamiento.

Algoritmo 13 $\hat{\theta} \leftarrow \text{entrenamientoD}(\mathbf{x}_{1:N_{data}}, \theta)$

/ Entrenamiento para predicción de siguiente token */*

Entrada: $\{\mathbf{x}_n\}_{n=1}^{N_{data}}$, un conjunto de datos de secuencias

Entrada: θ , valores de partida de los parámetros del transformador

Salida: $\hat{\theta}$, valor de los parámetros después del entrenamiento

- 1 **para** $i = 1, 2, \dots, N_{epochs}$ *hacer*
 - 2 **para** $n = 1, 2, \dots, N_{data}$ *hacer*
 - 3 $\ell \leftarrow \text{longitud}(\mathbf{x}_n)$
 - 4 $\mathbf{P}_\theta \leftarrow \text{transformador_de_decodificadores}(\mathbf{x}_n \mid \theta)$
 - 5 $\text{loss}(\theta) = -\sum_{t=1}^{\ell-1} \log P(\theta)[x_n[t+1], t]$
 - 6 $\theta \leftarrow \theta - \eta \nabla \text{loss}(\theta)$
 - 7 **fpara**
 - 8 **fpara**
 - 9 **retorna:** $\hat{\theta} = \theta$
-

Algoritmo 14 $\hat{\theta} \leftarrow \text{inferencia_transformadorD}(\mathbf{x}, \hat{\theta})$

/* Predicción usando un modelo entrenado */

Entrada: $\hat{\theta}$, parámetros entrenados**Entrada:** $\mathbf{x} \in V^*$, una secuencia de entrada**Salida:** $\gamma \in V^*$, predicción de la continuación de la secuencia de entrada**Hiperparámetros:** $\ell_{gen} \in \mathbb{N}, \tau \in (0, \infty)$

```

1   $\ell \leftarrow \text{longitud}(\mathbf{x})$ 
2  para  $n = 1, 2, \dots, \ell_{gen}$  hacer
3       $\mathbf{P} \leftarrow \text{transformador\_de\_decodificadores}(\mathbf{x} \mid \hat{\theta})$ 
4       $\mathbf{p} \leftarrow \mathbf{P}[:, \ell + i - 1]$ 
5      muestrea un token  $\gamma$  de  $\mathbf{q} \sim \mathbf{p}^{1/\tau}$ 
6       $\mathbf{x} \leftarrow [\mathbf{x}, \gamma]$ 
7  fpara
8  retorna  $\gamma = \mathbf{x}[\ell + 1 : \ell + \ell_{gen}]$ 
```

Algoritmo 15 $\hat{\theta} \leftarrow \text{inferencia_transformadorCD}(\mathbf{z}, \hat{\theta})$

/* Predicción usando un transformador secuencia a secuencia */

Entrada: $\hat{\theta}$, parámetros entrenados**Entrada:** $\mathbf{z} \in V^*$, una secuencia de entrada**Salida:** $\hat{\mathbf{x}} \in V^*$, predicción de la secuencia de salida**Hiperparámetros:** $\tau \in (0, \infty)$

```

1   $\hat{\mathbf{x}} \leftarrow [\text{bos\_token}]$ 
2   $\gamma \leftarrow 0$ 
3  mientras  $\gamma \neq \text{eos\_token}$  hacer
4       $\mathbf{P} \leftarrow \text{transformador\_CD}(\mathbf{z}, \hat{\mathbf{x}} \mid \hat{\theta})$ 
5       $\mathbf{p} \leftarrow \mathbf{P}[:, \text{longitud}(\hat{\mathbf{x}})]$ 
5      muestrea un token  $\gamma$  de  $\mathbf{q} \sim \mathbf{p}^{1/\tau}$ 
6       $\hat{\mathbf{x}} \leftarrow [\hat{\mathbf{x}}, \gamma]$ 
7  fpara
8  retorna  $\hat{\mathbf{x}}$ 
```


2.4 Transformadores para procesamiento del lenguaje natural (NLP)

2.4.1 Desarrollos previos: *embeddings*

Previo a la introducción de los algoritmos basados en transformadores es importante hacer mención a un conjunto de algoritmos que se crearon en los años anteriores, concretamente a los algoritmos de creación de *embeddings*.

En el contexto de NLP, un *embedding* es una representación matemática del significado de las palabras. Esta transformación parte del supuesto que es posible representar el conocimiento en un espacio vectorial multidimensional. De esta manera cualquier palabra del lenguaje tendrá una transformación en ese espacio. A los vectores representativos de las palabras se les denomina *embeddings*.

En los años anteriores a la introducción de los transformadores, se realizaron multitud de publicaciones definiendo distintos métodos para realizar dicha transformación. Entre los más conocidos tenemos a **Word2Vec** (Mikolov y otros, 2013), **GloVe** (Pennington y otros, 2014), **FastText** (Joulin y otros, 2016), entre otros.

Estos algoritmos realizan la transformación utilizando una red neuronal poco profunda. Entre las propiedades más destacadas de estos algoritmos están las siguientes:

- **Similitud entre palabras sinónimas o semejantes:** en el espacio transformado, palabras de significado similar convergen en ubicaciones similares. Así, por ejemplo, palabras como ‘coche’, ‘vehículo’ o ‘furgoneta’ están relativamente más cercanas entre ellas de otras como ‘luna’, ‘espacio’ o ‘árbol’, entendiendo la proximidad como una similitud calculada como distancia euclidiana o similitud del coseno, por ejemplo.
- **Codificación de relaciones lingüísticas entre palabras:** una característica cuando menos sorprendente de estas transformaciones es la codificación de algunas relaciones entre palabras como transformaciones lineales. Por ejemplo, la transformación lineal entre ‘hombre’ y ‘mujer’ es similar a la necesaria entre ‘rey’ y ‘reina’, ‘tío’ y ‘tía’ o ‘actor’ y ‘actriz’, generalizando por tanto esa transformación como una transformación de género. Así, esto habilita la posibilidad de realizar operaciones con resultados aproximados como $\vec{\text{rey}} - \vec{\text{hombre}} + \vec{\text{mujer}} \approx \vec{\text{reina}}$ o como $\vec{\text{París}} - \vec{\text{Francia}} + \vec{\text{Alemania}} \approx \vec{\text{Berlín}}$

Las limitaciones de estos métodos proceden de la propia naturaleza de su diseño. Al estar entrenados para codificar el significado de las palabras aisladas, no codifican el contexto. Una posibilidad para integrar el contexto, en el entorno

de las redes neuronales, pasó por utilizar los *embeddings* como transformaciones previas a la entrada a las redes neuronales recurrentes, con el objetivo de que estas se encargaran de codificar el contexto como medio de resolución de problemas de procesamiento natural concretos.

Los transformadores suponen un paso más allá. El objetivo de los mismos pasa por realizar un aprendizaje de inicio a fin. Las transformaciones del *embedding* pasan a formar parte de la propia red y son aprendidas en tiempo de entrenamiento, quedando integradas como parte de la red global como los parámetros de autoatención de la primera capa.

2.4.2 Inicios de los transformadores para NLP

Las publicaciones iniciales que marcaron la evolución posterior de los transformadores se circunscriben en el ámbito del procesamiento natural. Presentamos a continuación, en orden cronológico, un breve resumen de aquellas que consideramos más relevantes para la definición de este paradigma de diseño, así como las aportaciones más significativas de cada una de estas publicaciones. Una introducción de la arquitectura puede verse en Vaswani y otros (2017):

- **ULMFiT** (Universal Language Model Fine-tuning for Text Classification). Los autores presentan una solución para resolver cualquier tarea de NLP mediante aprendizaje por transferencia. Introdujo la idea de *general-domain*, aprendizaje no supervisado seguido de *fine-tuning*. Introdujeron también otras técnicas ampliamente utilizadas posteriormente en otros trabajos como «Slanted triangular learning rate schedules (akas warm-up)» (Howard y Ruder, 2018).
- **GPT** (Radford y otros, 2018) y **GPT-2** (Radford y otros, 2019) (Generative Pre-Trained Transformers). Se presenta una arquitectura formada únicamente por decodificadores. La mayor contribución de estas publicaciones es que demostraron el valor de entrenar modelos de enorme tamaño para la predicción de la palabra siguiente que se presenta en un texto.
- **BERT** (Devlin y otros, 2018) (Bidirectional Encoder Representations from Transformers.) Los autores usan únicamente codificadores para preentrenar representaciones bidireccionales de texto no etiquetado. Estos modelos son afinados posteriormente con un segundo entrenamiento, añadiendo únicamente una capa de salida adicional, para resolver aplicaciones específicas. Este modelo supuso un antes y un después en la resolución de problemas de NLP.
- **RoBERTa** (Liu y otros, 2019) (Robustly Optimized BERT Approach). Los autores proponen una arquitectura idéntica a BERT introduciendo una serie

de mejoras del procedimiento de entrenamiento. Básicamente, es un BERT con un entrenamiento mejorado.

- **T5** (Raffel y otros, 2020) (Text-to-Text Transfer Transformer). Las principales contribuciones de este trabajo son, por una parte, la reformulación de la forma de reportar los resultados sustituyendo el enfoque numérico por un enfoque textual y, por otra, el estudio sistemático de la contribución de la función objetivo, arquitecturas, conjuntos de datos, *transfer approaches* y otros factores en los resultados obtenidos en un conjunto de tareas canónicas de NLP.

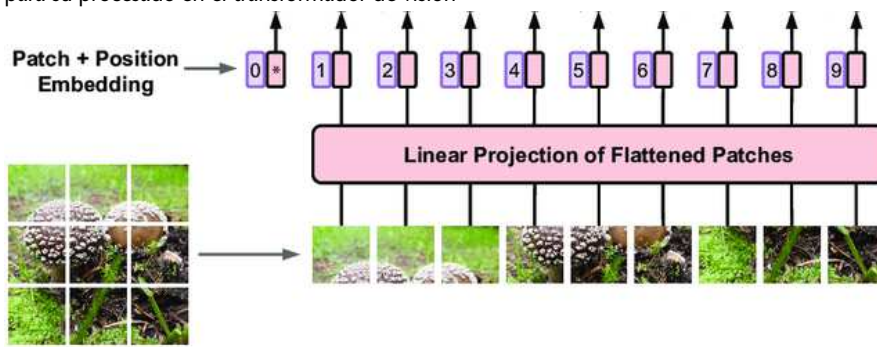
2.5 Transformadores para Visión (ViT)

En los subapartados anteriores hemos estudiado los transformadores en su aplicación de referencia, el procesamiento del lenguaje natural. Después de los prometedores resultados obtenidos en dicho campo, algunos autores empezaron a considerar la posibilidad de usar dicha arquitectura en otros campos como la visión.

Desde su introducción, se demostró que el rendimiento de los modelos ViT es comparable al de arquitecturas especializadas como son las redes convolucionales para aplicaciones típicas de visión por computador, como la clasificación de imágenes (Dosovitskiy y otros, 2020), detección de objetos (Zhu y otros, 2020), segmentación semántica (Zheng y otros, 2021b), colorización de la imagen (Kumar y otros, 2021), visión de bajo nivel (Chen y otros, 2021a) y comprensión de vídeo (Arnab y otros, 2021), por citar algunas. Además, recientes investigaciones indican que los errores de predicción de ViT son más consistentes con los de los humanos que las redes convolucionales (Tuli y otros, 2021).

El funcionamiento de los ViT es similar a un transformador convencional. La única diferencia esencial radica en la naturaleza de la señal de entrada, que en vez de ser una secuencia de palabras, es una imagen. La problemática radica, por tanto, en cómo tratar la entrada para que sea gestionable por el modelo; esto es, adaptar el *tokenizador* para trabajar con imágenes. Una de las primeras propuestas realizadas en la bibliografía, con muy buenos resultados, consiste en particionar la imagen en partes disjuntas más pequeñas y alimentarlas a la red de forma secuencial. Cada parte tiene las mismas dimensiones (16x16, por ejemplo en Dosovitskiy y otros (2020)) y actúa de forma equivalente a lo que sería la codificación de un elemento en una secuencia. A este vector se le añade un *embedding* para codificar la posición que ocupa en la imagen, al estilo de lo que se realiza con las secuencias de texto para ubicar cada elemento.

Figura 14. Diagrama del proceso de partición, secuenciación y embedding de una imagen para su procesamiento en el transformador de visión



Fuente: Adaptado de Pícek y otros (2022)

En Liu y otros (2021) se puede encontrar un exhaustivo compendio de todas las referencias existentes hasta el momento relativas a transformadores para visión. Si bien el objetivo principal de la arquitectura de los transformadores es conseguir un tratamiento generalista para datos heterogéneos, no todas las publicaciones existentes van en ese sentido. Así, podemos encontrar artículos que modifican los sistemas de atención para conseguir las propiedades de localidad semejantes a las de las redes convolucionales mediante el uso de atención local o jerárquica, conceptos que describiremos en el apartado dedicado a los transformadores multimodales. Otras utilizan como entrada del transformador atributos representativos de la imagen procedentes de redes convolucionales preentrenadas. Aunque se aleja del objetivo de este módulo el presentar la miríada de variaciones existentes publicadas que están alejadas del espíritu generalista de la arquitectura, realizaremos una presentación general de las propuestas realizadas hasta el momento al respecto. Hay que tener en cuenta que los transformadores, en general, y para visión en particular, son una arquitectura novel, por tanto no madura, que seguro que estará sujeta a grandes innovaciones futuras. Independientemente del interés que pueda suscitar una aplicación particular, el objetivo de este módulo está destinado a explicar aquellas arquitecturas más generalistas que funcionen bien en un amplio abanico de circunstancias. En los subapartados que siguen intentaremos presentar aquellas arquitecturas que cumplen ese cometido para las aplicaciones de visión en los campos de clasificación, detección y segmentación. Para aplicaciones más específicas se puede consultar la extensa bibliografía.

2.5.1 Clasificación

Hasta el momento se puede decir que hay hasta cuatro formas distintas de abordar la contribución de los transformadores en la resolución de problemas de clasificación, y que enumeramos a continuación:

1) Adaptación directa de la arquitectura original (Dosovitskiy y otros, 2020). La imagen original se dispone como un conjunto de partes más pequeñas no superpuestas (*patches*), que son tratadas como elementos de una secuencia.

2) Transformador de atributos procedentes de una red convolucional (Wu y otros, 2020). Los atributos extraídos de una capa intermedia de una red CNN preentrenada son alimentados a un transformador para mejorar sus capacidades de representación. La ventaja de este enfoque es que los elementos de la secuencia inicial ya separan y codifican la información, que es combinada y perfeccionada por el transformador. De esta manera se aprovechan los priores estructurales de la CNN para facilitar el trabajo posterior.

3) Destilación del conocimiento de una convolucional preentrenada (Touvron y otros, 2021). Una red CNN actúa como maestra de una red basada en transformadores. El proceso de aprendizaje consigue transferir los sesgos inductivos a la red aprendiz mediante destilación del conocimiento (Hinton y otros, 2015). Sorprendentemente, la red aprendiz basada en transformadores acaba obteniendo mejores resultados predictivos que los de la red convolucional de la que deriva.

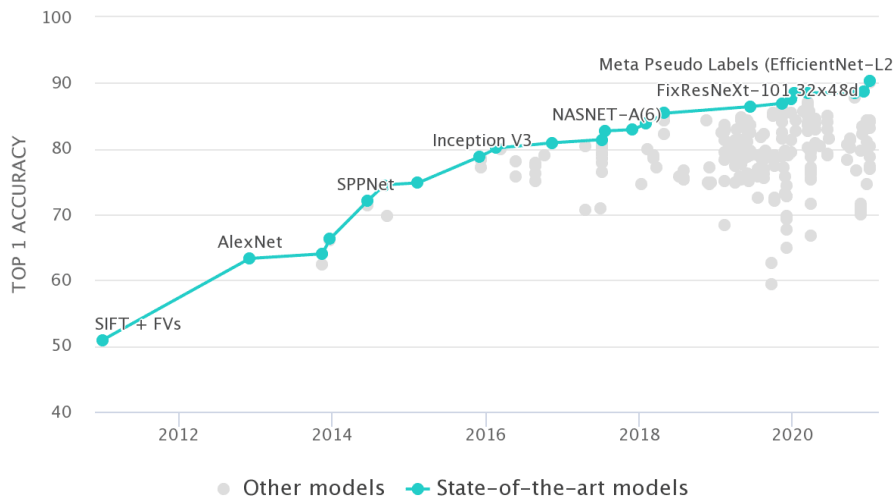
4) Transformador con atención localizada (Liu y otros, 2021) y **transformadores jerárquicos**. Es un transformador con módulos de atención adaptados para funcionar de forma localizada. A partir de la partición de la imagen de entrada en elementos más pequeños que en ViT original (Dosovitskiy y otros, 2020) (4x4 en vez de 16x16), estos se alimentan a matrices de autoatención que combinan únicamente las particiones más próximas. Una vez calculada la atención, se recombina con las atenciones circundantes mediante una capa de fusión. Las capas siguientes aplican el mismo modelo de atención localizada desplazando las particiones combinadas. De esta forma se consigue combinar la información con las zonas adyacentes. En Yuan y otros (2021) y en Wang y otros (2021) se utilizan aproximaciones similares basadas en un tratamiento jerárquico donde los módulos de atención son modificados para integrar una progresiva reducción del espacio de cálculo.

Destilación del conocimiento

Conocido en inglés como *Knowledge Distillation*, es el proceso de transferencia del conocimiento de una red origen a otra destino sin la necesidad de datos externos. A efectos de aprendizaje, la red de origen actúa a modo de maestro, mientras que la de destino actúa como alumno. Habitualmente, la red de origen es mayor que la de destino y ha sido entrenada previamente a partir de un conjunto de datos que no interviene en el proceso.

En la figura 15 se muestra la evolución temporal del estado del arte para clasificación de imágenes (Benchmark Imagenet). Se puede observar como a mediados de 2022 los modelos con mejor rendimiento estaban basados en transformadores (**ViT-G/14**, 1843M de parámetros, 91.0% (Ding y otros, 2022)), seguidos muy de cerca de uno basado en redes convolucionales (**EfficientNet-L2**, 480M de parámetros, 90.2% (Pham y otros, 2021)). Para hacer comparativos los resultados, buscamos un transformador con un número similar de parámetros. Encontramos que **DaViT-H**, con 362M consigue también un 90.2% (Ding y otros, 2022). A partir de estos resultados se puede concluir que la arquitectura de los transformadores, a pesar de no disponer de priores estructurales, consigue obtener resultados de clasificación parejos e incluso mejores de rendimiento respecto a las CNN más competitivas, con una complejidad computacional similar. En contrapartida cabe indicar que para conseguir estos resultados es necesario someter a las redes de transformadores a procesos de optimización más prolongados y con conjuntos de datos grandes. Una alternativa cuando no se dispone de suficientes datos etiquetados es utilizar una etapa previa de aprendizaje semisupervisado (Weng y otros, 2021).

Figura 15. Estado del arte de los modelos de clasificación de imagen



Fuente: ima

2.5.2 Detección

Cuando referimos al uso de transformadores para detección en imágenes, debemos realizar una distinción de inicio entre las aplicaciones que utilizan un enfoque de transformador de extremo a extremo, de aquellas que lo usan únicamente como *backbone* para la extracción de atributos, pero utilizan un bloque final de detección convencional (por ejemplo, **RetinaNet** (Lin y otros, 2017b) o **Mask R-CNN** (He y otros, 2017)).

Los *backbone* mencionados vienen a ser transformadores preentrenados en tareas de clasificación que se utilizan como extractor de atributos para alimentar a un cabezal de detección.

En lo que respecta a los transformadores de extremo a extremo, en el momento de la redacción de este apartado existen dos paradigmas distintos para abordar el problema de detección: uno sería la familia de modelos derivados de la arquitectura DETR (Carion y otros, 2020) y el otro basado en modelos derivados de Pix2Seq (Chen y otros, 2021b). Estudiamos a continuación las características definitorias de cada uno de ellos.

DETR: DETection with TRansformer

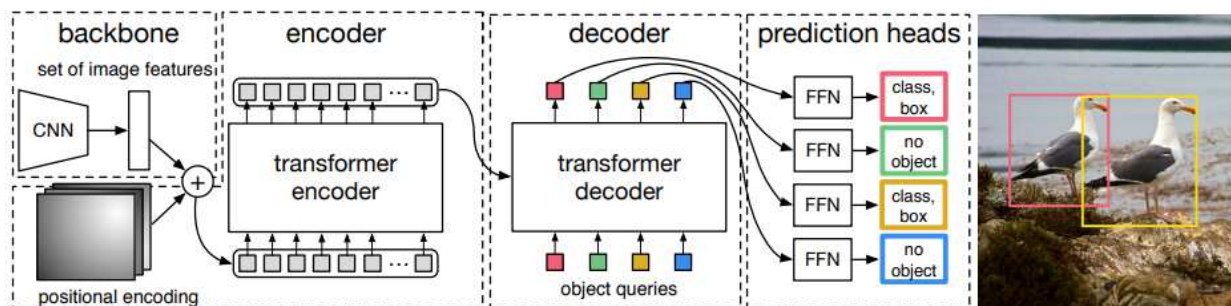
En la figura 16 se muestra esquemáticamente la arquitectura de detección DETR. La imagen de entrada se particiona, secuencia y alimenta a un transformador secuencia a secuencia. El *tokenizador* está formado por una red convolucional que extrae los atributos de la imagen original para posteriormente entrar en el transformador codificador. La originalidad de esta propuesta viene en la parte de decodificación. Esta, por un lado, recibe los datos procedentes

Bibliografía recomendada

Para un compendio de los modelos que utilizan este tipo enfoque se puede consultar a Liu y otros (2021).

de la codificación y, por otro, una secuencia de entrada. Esta representa la propuesta de objetos a detectar. A su paso por la red de decodificación se combina con la información de la imagen, dando como resultado la secuencia de salida. Si la red ha cumplido su función, la secuencia de salida contendrá la representación interna de la detección de los objetos presentes en la imagen. La secuencia de elementos de salida tiene la misma cardinalidad que la de entrada y alimenta a una red neuronal independiente para cada elemento. De esta manera, se deriva la clase y el cuadro delimitador de los elementos detectados. El tamaño de la secuencia del decodificador determina el número máximo de elementos que es posible detectar. En el artículo original, el número máximo de objetos es de 100. El modelo prevé una clase especial para indicar la no presencia de objeto para cada posición. En **DETR** la secuencia de entrada está formada por números aleatorios que sirven como prior para la detección de los objetos. A pesar de la naturaleza aleatoria del prior de **DETR** los resultados obtenidos son muy buenos. Trabajos publicados posteriormente han intentado mejorar las capacidades predictivas de la red afinando los priores de la secuencia de entrada (**SMCA** (Gao y otros, 2021), **Conditional DETR** (Meng y otros, 2021), **Anchor DETR** (Wang y otros, 2022), **DAB-DETR** (Liu y otros, 2022), **Efficient DETR** (Yao y otros, 2021), **Dynamic DETR** (Dai y otros, 2021)).

Figura 16. Arquitectura del transformador **DETR** especializado en detección



Fuente: Carion y otros (2020)

Un punto clave del diseño de esta arquitectura es el diseño de una función de pérdida que permita la optimización eficiente de la red durante el aprendizaje supervisado, es decir, la comparación de los valores predichos con los reales. Para comprender la dificultad del problema, podemos ver el ejemplo indicado en la tabla 3. Observamos como las clases predichas y reales no necesariamente se van a encontrar en la misma posición. Además, el ordenamiento de las predicciones tampoco garantiza la coincidencia, pues será frecuente el caso en que el modelo no sea capaz de predecir todos los elementos o que lo haga de forma errónea. Este problema se resuelve calculando la función de pérdida para todas las agrupaciones posibles entre valores verdaderos y predichos, escogiendo aquella combinación donde la pérdida es menor (Carion y otros, 2020), (Stewart y otros, 2016).

Tabla 3. Ejemplo de valor real y predicho en una red de detección

Real			Predicho		
Clase	Desde	Hasta	Clase	Desde	Hasta
vaso	(100, 200)	(200, 300)	sin objeto	(20, 40)	(10,0)
botella	(300, 200)	(400, 300)	vaso	(98, 198)	(210,310)
bol	(100, 200)	(200, 300)	sin objeto	(20, 40)	(10,0)
portátil	(100, 200)	(200, 300)	sin objeto	(20, 40)	(10,0)
ratón	(100, 200)	(200, 300)	sin objeto	(20, 40)	(10,0)
silla	(500, 400)	(700, 500)	bolsa	(460, 560)	(510, 599)
bolsa	(450, 550)	(500, 600)	sin objeto	(20, 40)	(10,0)
sin objeto	(0, 0)	(0, 0)	sin objeto	(20, 40)	(10,0)
sin objeto	(0, 0)	(0, 0)	botella	(20, 40)	(10,0)
sin objeto	(0, 0)	(0, 0)	sin objeto	(20, 40)	(10,0)

La función de pérdida calculada con el emparejamiento bipartito es la que se muestra en la ecuación 6.

$$\hat{\sigma} = \underset{\sigma \in \mathcal{G}_N}{\operatorname{argmin}} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}), \quad (6)$$

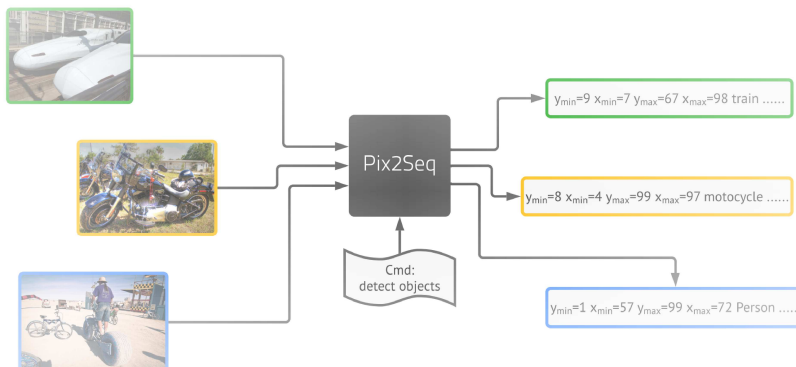
$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) = -\mathbb{1}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

Una vez hecho el emparejamiento óptimo, para llevar a cabo la optimización de la red se utiliza la función de pérdida húngara (ecuación 7).

$$\mathcal{L}_H(y_i, \hat{y}_{\sigma(i)}) = \sum_{i=1}^N [-\log \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})] \quad (7)$$

Pix2Seq

En la figura 17 se muestra un esquema del funcionamiento de **Pix2Seq**. Utiliza un nuevo paradigma consistente en un mapeo directo entre la imagen a tratar y la salida de secuencia de texto, donde se codifican clase y cuadro delimitador de todos los elementos detectados. Es una solución elegante con resultados de rendimiento similares a los obtenidos por **DETR** e incluso en algunas circunstancias mejores (por ejemplo, en la detección de objetos pequeños).

Figura 17. Tipología de E/S del transformador **Pix2Seq** especializado en detección

Utiliza una red de transformadores formada por codificadores y decodificadores resolviendo de forma autorregresiva. La parte de codificación puede ser o bien un transformador o una red convolucional. El codificador extrae los atributos de interés de la imagen para alimentarlos a la red de decodificadores, que los convierten junto con los comandos procedentes de la secuencia de entrada en una salida textual. Se genera un elemento cada vez, condicionado a los atributos suministrados por el codificador y a los elementos de la secuencia generados hasta el momento.

La naturaleza textual de la salida simplifica los cálculos requeridos de la función de pérdida con respecto a DETR. El formato esperado de la salida es un conjunto de elementos de la forma $[x_min], [y_min], [x_max], [y_max], [clase], \dots, [EOS]$, donde EOS sería el indicador de final de secuencia que aparece después de la salida de todos los «cajetines» delimitadores presentes en la imagen. Durante el entrenamiento, la ordenación de los elementos en la secuencia de predicción se hace de forma aleatoria. Se optimiza mediante la función de pérdida típica de *log-likelihood* utilizada por los transformadores autorregresivos de NLP (ecuación 8).

$$\max \sum_{j=1}^L \omega_j \log P(\tilde{y}_j | \mathbf{x}, \mathbf{y}_{1:j-1}) \quad (8)$$

donde $\mathbf{x}$ es una imagen determinada, $\mathbf{y}$ e $\tilde{\mathbf{y}}$ las secuencias de entrada y objetivo, respectivamente, asociadas con $\mathbf{x}$ y L la longitud de la secuencia. ω_j es un peso preestablecido para el elemento j de la secuencia. En el caso de Pix2Seq, $\omega_j = 1, \forall j$

2.5.3 Segmentación

Segmentación es el término genérico que se utiliza para referirse en realidad a tres problemas distintos: **segmentación semántica**, **segmentación de instancias** y **segmentación panóptica**. La **segmentación semántica** es el problema consistente en asignar a cada píxel de una imagen una clase. Este tipo de segmentación es el más sencillo, ya que no requiere de la separación entre objetos distintos pertenecientes a una misma clase. En la **segmentación panóptica** y en la **segmentación de instancias** el objetivo es más complejo, porque no únicamente pretenden separar clases, sino también objetos. La diferencia entre ambas es que la panóptica identifica cada píxel con una única clase, mientras que la de instancias permite que un píxel esté asociado a más de una clase, teniendo en cuenta la superposición de objetos.

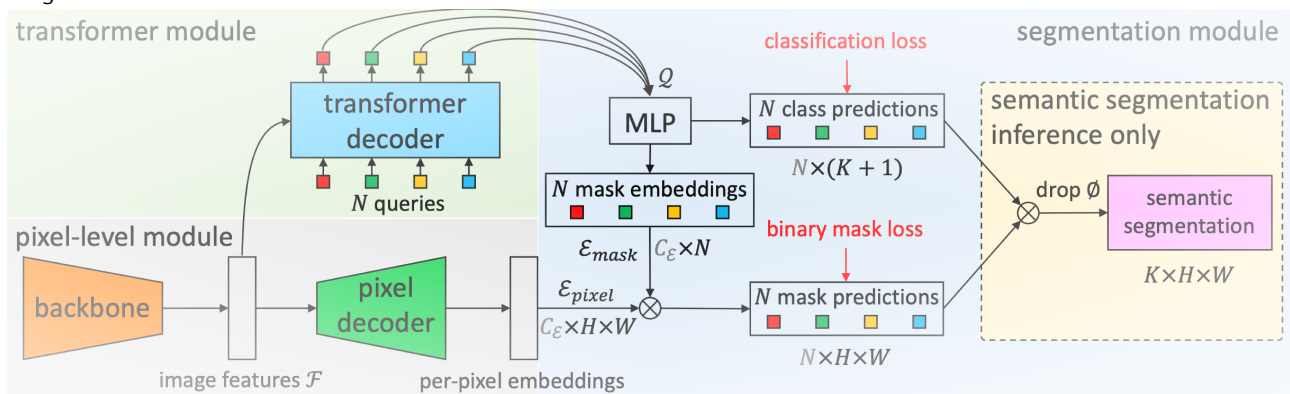
Las propuestas para resolver los problemas de segmentación utilizando transformadores han estado y están sometidas a una profunda evolución. Podemos distinguir entre dos enfoques: por un lado, la segmentación directa a partir de la información de los píxeles de la imagen y, por otro, la construcción de

representaciones internas basadas en objetos, para inferir posteriormente las máscaras de segmentación.

Las propuestas iniciales basadas en el método directo son una extensión del ViT para la resolución de tareas de segmentación (véase **SETR** (Zheng y otros, 2021b)). Esta primera aproximación sirvió para demostrar la viabilidad de la arquitectura para la resolución de este tipo de problemas, aunque con unos costes computacionales elevados. Posteriormente, se presentaron propuestas basadas en la representación interna de objetos con arquitecturas derivadas de la arquitectura de detección **DETR** presentada en el subapartado anterior. Las primeras propuestas requerían de clase, cuadro delimitador y máscara. Las posteriores son capaces de derivar máscara y clase directamente sin la necesidad de cuadro delimitador. Presentaremos en este subapartado la arquitectura **Mask2Former** (Cheng y otros, 2022), como un ejemplo de una arquitectura madura que permite resolver con una única arquitectura las tres tareas de segmentación: semántica, de instancias y panóptica.

En la figura 18 se muestra el esquema representativo del transformador de segmentación **MaskFormer** (válido también para **Mask2Former**).

Figura 18. Transformador MaskFormer. Arquitectura especializada en segmentación de imágenes



Fuente: Cheng y otros (2021)

Podemos observar que consta de un *backbone*, que puede ser tanto una red convolucional o como un ViT, del que se extraen los atributos representativos de la imagen. Estos se alimentan a un decodificador de píxeles y a un transformador decodificador. El decodificador de píxeles es una FPN (Feature Pyramid Network) (Lin y otros, 2017a), pero podría ser cualquier otra arquitectura que sea capaz de transformar atributos a imagen (un ejemplo alternativo podría ser una U-NET (Ronneberger y otros, 2015)). El transformador decodificador, al estilo del **DETR**, genera una secuencia de salida a partir de una secuencia de entrada utilizando la representación interna de los objetos presentes en la imagen procedente del *backbone*. A partir de la misma, un MLP que genera la predicción de clases para cada objeto, así como una representación interna de la máscara, que, combinada con la información procedente de la salida

del decodificador de píxeles, da como resultado las predicciones de las máscaras para cada uno de los objetos. Al igual que con DETR, existe una clase especial para identificar aquellas clases en las que no se ha detectado objeto y para las que, por tanto, la segmentación reportada no debe tenerse en cuenta. El número máximo de máscaras que la red es capaz de detectar es un valor de diseño de la arquitectura y está limitado por el tamaño de la secuencia de objetos.

2.6 Transformadores para audio

Los transformadores también se están convirtiendo en el estándar *de facto* para el tratamiento de señales de audio, espacio hasta el momento ocupado por las redes convolucionales y anteriormente por las redes recurrentes. Las posibilidades en este campo son muy amplias y diversas. La arquitectura usada es la misma que en las aplicaciones anteriormente presentadas para otro tipo de señales, siendo el hecho distintivo el *tokenizador* utilizado para transformar la señal de entrada en una representación interna manejable por la red.

2.6.1 Señales de audio, tratamiento y tokenización

Las entradas de texto e imagen son tipos de datos más intuitivos de representar. Para el caso del sonido, parece más complejo el modo de expresión numérica. Realizaremos una explicación de la naturaleza de las señales de audio para intentar comprender mejor cómo se realiza la preparación para su utilización en aplicaciones de predicción automática.

Una señal viene a ser la evolución temporal de una variable física que puede ser medida. En el caso del audio, estaríamos refiriéndonos a variaciones de la presión del aire en un determinado punto. Para tratar digitalmente estas señales, primero hay que almacenarlas. Esto se hace inicialmente capturando esta información temporal realizando un muestreo, esto es, registrando el valor de la propiedad física con una frecuencia preestablecida. Esta frecuencia de muestreo puede variar, pero un valor habitual para sonido puede ser, por ejemplo, 44.1 kHz, esto es, 44100 muestras por segundo. Una vez capturada esta información, se dispone de la representación digital de la señal de audio y está en condiciones de ser tratada digitalmente.

Del análisis matemático conocemos el teorema de Fourier, que viene a decir que toda señal periódica definida en un intervalo temporal finito, independientemente de su forma, puede ser aproximada por la suma de señales senooidales. Cada una de estas señales constituyentes está definida por únicamente dos variables: amplitud y frecuencia. Dichas variables pueden ser mostradas gráficamente en lo que se denomina un espectro de frecuencias. Así, es posible representar la misma señal de dos formas distintas: atendiendo al dominio

temporal o al dominio de frecuencia. A la transformación entre un dominio y el otro se denomina transformada de Fourier (Bracewell y Bracewell, 1986). El algoritmo FFT (Fast Fourier Transform) (Nussbaumer, 1981) permite llevar a cabo de manera eficiente dicha transformación en un entorno digital.

Lo presentado hasta ahora aplica a señales periódicas, pero el habla o la música no son periódicas, ¿qué ocurre entonces en estos casos? La solución pasa por definir una longitud de ventana en la que se trata la señal como periódica y para la cual se calcula su espectro. Llevando a cabo este cálculo para ventanas consecutivas superpuestas, obtenemos una representación en dominio de frecuencia de la señal no periódica. A este gráfico se le denomina espectrograma. El espectrograma es una representación gráfica de la frecuencia respecto del tiempo. Cada punto del mismo representa la amplitud en color en decibelios. Los decibelios vienen a representar la amplitud en escala logarítmica. Se utiliza dicha escala debido a que la percepción de los cambios de sonido en humanos está más alineada con ese orden de escala.

Al igual que pasa con la amplitud, los humanos no percibimos las frecuencias de manera lineal. Nuestro oído está mejor adaptado a percibir las diferencias para bajas frecuencias (sonidos graves). Una persona sana es capaz de distinguir sin problemas la diferencia entre una señal a 500 Hz de una de 1000 Hz, pero le será mucho más difícil hacerlo entre una de 10 000 Hz y una de 10 500 Hz, a pesar de que la diferencia entre ambas es la misma. En 1937, se propuso la escala de MEL (Stevens y otros, 1937), que pretendía linealizar la percepción de la diferencia de frecuencias introduciendo una nueva unidad denominada *pitch* de MEL. El gráfico resultante de representar la amplitud en decibelios en un gráfico bidimensional de *pitch* vs. tiempo es más representativo, linealmente, de la percepción del sentido del oído en el ser humano. A este gráfico se le denomina espectrograma de MEL. Suele ser el espectrograma más habitual en los modelos de aprendizaje automático. La linealidad del gráfico con la percepción auditiva humana enfatiza la información relevante y facilita el procesamiento posterior de la señal.

Llegados a este punto podemos entender que una de las representaciones más habituales de las señales de audio para el aprendizaje automático sea el espectrograma de MEL. Al ser una matriz de dos dimensiones que suele tener un tamaño mayor del gestionable directamente por el transformador, un *tokenizador* habitual (otros también son posibles) consiste en aplicar convoluciones que reduzcan su tamaño al espacio interno de la arquitectura.

2.6.2 Ejemplos de transformadores de audio

A continuación se presenta una muestra de las muchas otras aplicaciones de audio que utilizan transformadores:

- **Whisper** (Radford y otros, 2022). Es un transformador secuencia a secuencia multilingüaje que, a partir de una fuente de audio que contenga una conversación, devuelve la secuencia de texto asociada.
- **Wav2Vec 2.0** (Baevski y otros, 2020). Es un transformador con una finalidad similar a BERT, pero que en vez de utilizar secuencias de texto trabaja directamente con secuencias de audio. Utiliza una red convolucional como codificador de atributos que posteriormente se alimentan al transformador. Realiza un aprendizaje no supervisado para obtener representaciones internas de secuencias de audio. El modelo puede ser posteriormente afinado con relativamente pocos datos para resolver problemas específicos.
- **SepFormer** (Subakan y otros, 2021). Es una red de transformadores especializada en la separación de voces de una fuente de audio.

2.7 Transformadores multimodales

Hasta ahora hemos visto la potencialidad de los transformadores para el aprendizaje automático utilizando datos de una única naturaleza. Debido a sus capacidades generalistas para resolver problemas de áreas hasta ahora inconexas, es lógico hipotetizar que dicha arquitectura potencialmente puede ser capaz de gestionar problemas que requieran entradas de distinta naturaleza al mismo tiempo. Un ejemplo claro de agente de este tipo son los animales y las personas, que integran información procedente de hasta cinco sentidos distintos.

Las arquitecturas multimodales son aquellas capaces de gestionar entradas de distinta naturaleza, por ejemplo, imagen y texto.

Desde el punto de vista de la geometría topológica, el espacio generado por la arquitectura de los transformadores es equivalente a la de un grafo completamente conectado (Bronstein y otros, 2021). Esto no es así en otras arquitecturas, donde las interconexiones del grafo están limitadas a su espacio más restringido. Los transformadores, por tanto, disponen de un espacio de modelado más general y flexible (Xu y otros, 2022). Esto representa una ventaja cuando se requiere combinar entradas de distinta naturaleza. Los priores estructurales que pueden ser útiles para facilitar la optimización para un determinado tipo de datos pueden ser un inconveniente cuando se usan junto con otras entradas de distinta naturaleza.

A continuación se describen los puntos diferenciales que existen entre una arquitectura multimodal y la tradicional.

2.7.1 Entradas multimodales

La primera función que realiza un transformador es la conversión de los datos de entrada en un tensor de dimensión interna definida, que se mantiene constante en su avance por las distintas capas. La *tokenización* transforma los elementos de entrada en un vector intermedio al que posteriormente se le aplica una transformación matemática que, normalmente, reduce sus dimensiones con el objetivo de alcanzar las internas de proceso. En entradas textuales es habitual utilizar un *tokenizador* que convierte cada elemento de la cadena de entrada en un vector ortogonal de dimensión igual a la del diccionario. Esta operación es seguida de una transformación lineal con el objetivo de alcanzar la dimensión interna de trabajo. En el caso de imagen, una estrategia consiste en *tokenizar* mediante la partición de la imagen en partes no superpuestas de tamaño predefinido, seguido de una transformación lineal de la misma para alcanzar la dimensión de trabajo (Dosovitskiy y otros, 2020).

Otro tipo de estrategias de *tokenización* consiste en el uso de redes especializadas para derivar atributos que son alimentados al transformador (Lu y otros, 2019). En el caso del vídeo, existen diversas estrategias: en Sun y otros (2019) la *tokenización* consiste en seleccionar clips de duración determinada, a unos fps preestablecidos, a los que seguidamente se les pasa a través de una red convolucional tridimensional para generar unos atributos que son posteriormente adaptados hasta conseguir las dimensiones internas de trabajo. Otras estrategias publicadas incluyen la *tokenización* mediante selección en los clips de puntos tridimensionales de interés, seguida por una transformación lineal (Akbari y otros, 2021); *tokenización* mediante la partición de las imágenes integrantes aplicada al conjunto del clip seguida de una proyección lineal (Nagrani y otros, 2021); entre otras. En el caso del audio, se pueden utilizar también distintas estrategias que dependen en parte del objetivo que se pretende conseguir. Una de ellas consiste, como hemos descrito anteriormente, en utilizar el espectrograma MEL (u otros derivados) seguido de una proyección lineal o del cálculo de atributos mediante una red convolucional (Lin y otros, 2021), (Akbari y otros, 2021), (Nagrani y otros, 2021).

Lectura recomendada

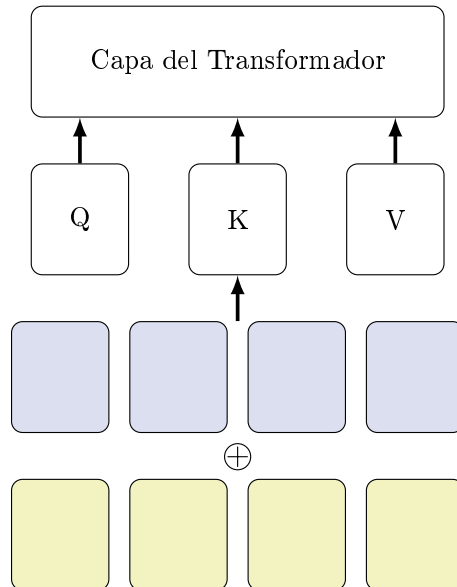
En Xu y otros (2022) se pueden encontrar ejemplos de módulos preparatorios para otro tipo de datos más específicos, como podrían ser esquemas de bases de datos SQL, nubes de puntos tridimensionales, datos tabulares, poses humanas, registros electrónicos de salud, conjunto de órdenes posibles a las que responde un robot, entre otros muchos.

2.7.2 Variantes de autoatención en un contexto multimodal

Una vez se dispone de la representación interna de las distintas señales de entrada, se plantea el dilema de cómo encontrar el modo más efectivo de combinarlas en la capa de autoatención para obtener los mejores resultados. A pesar de que puedan tener las mismas dimensiones, debido a su diferente naturaleza, puede ser necesario adaptar las transformaciones a cada uno de los distintos modos. Las estrategias más habituales para calcular la autoatención en esta primera etapa son: la suma temprana, la concatenación temprana, la jerárquica y la cruzada.

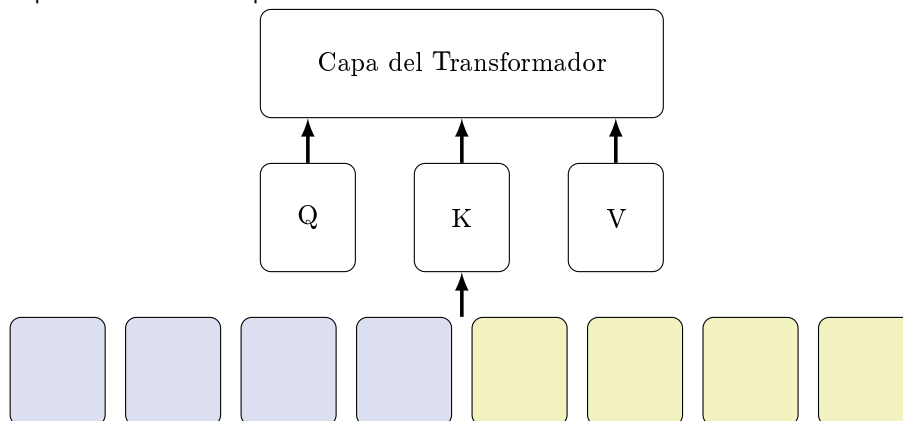
La **suma temprana** (figura 19) consiste en la suma de los vectores procedentes de las distintas fuentes como paso previo a un único sistema de autoatención común.

Figura 19. Atención por suma temprana. La información procedente de dos canales distintos se suma antes de ser procesada por el canal de autoatención.



La **concatenación temprana** (figura 20) genera un vector como concatenación de los distintos modos que se alimenta al módulo de autoatención.

Figura 20. Atención por concatenación. La información procedente de dos canales distintos se procesa en la misma capa de autoatención.



La **autoatención jerárquica** (figuras 21 y 22) se realiza en dos pasos, en uno se aplican transformaciones distintas a cada señal y en el otro una única que las combina.

Figura 21. Atención jerárquica multicanal a monocanal. La información procedente de canales distintos se procesa en una primera etapa en canales de atención independientes para posteriormente procesarse en conjunto en la etapa final.

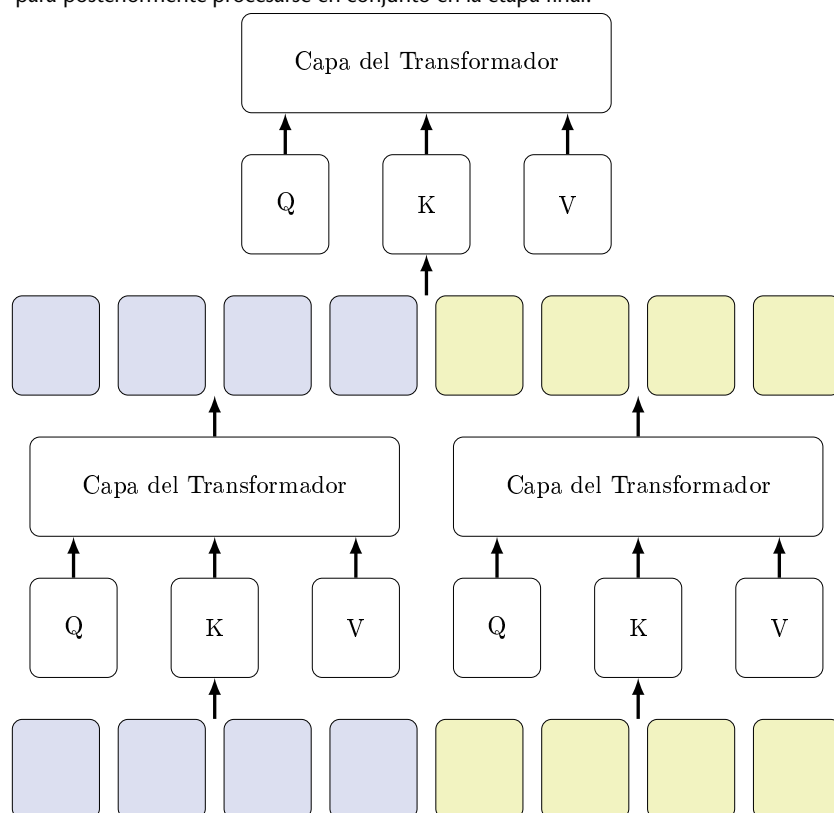
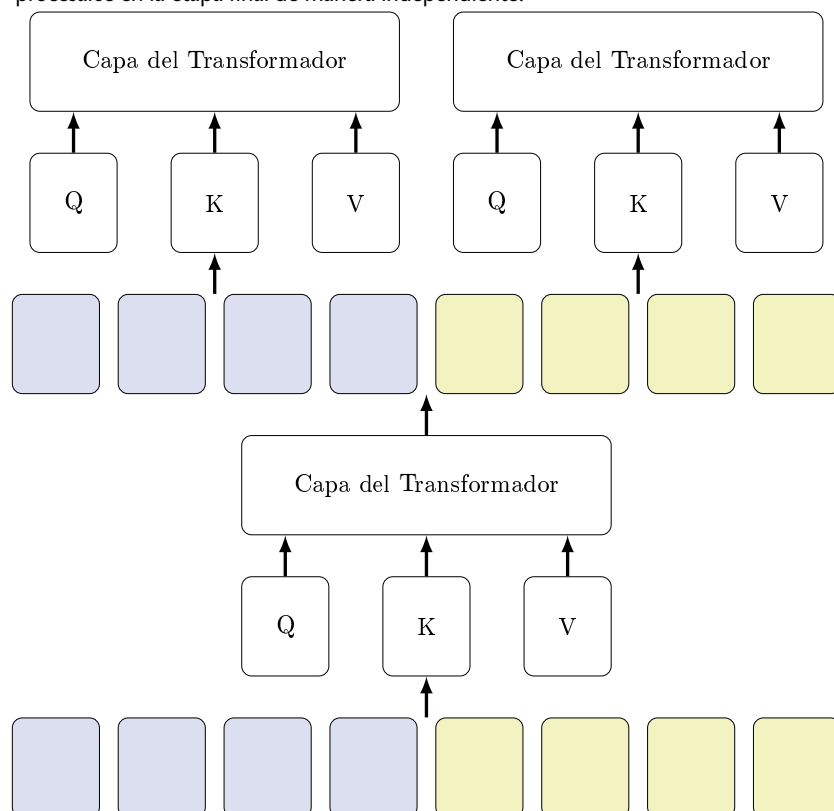
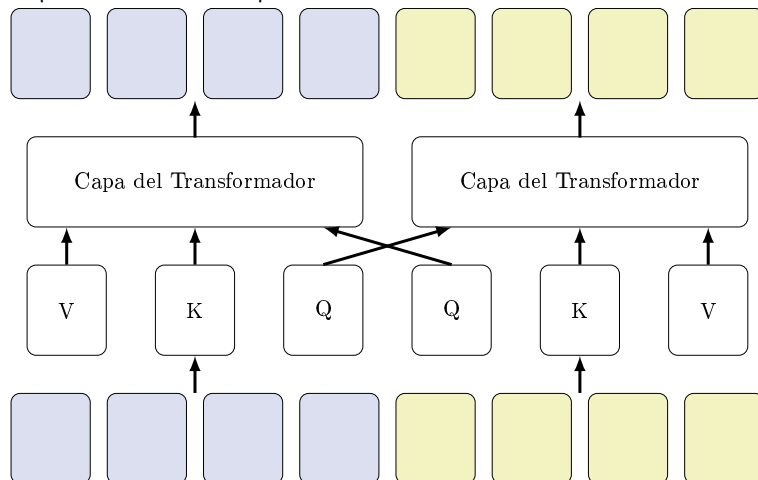


Figura 22. Atención jerárquica monocanal a multicanal. La información procedente de canales distintos se procesa en una primera etapa en conjunto para posteriormente procesarse en la etapa final de manera independiente.



La **autoatención cruzada** (figura 23) utiliza canales de autoatención independientes donde la matriz de Query de los canales actúa sobre el otro canal y no sobre el propio.

Figura 23. Atención cruzada. La información procedente de dos canales distintos se procesa en la misma capa de autoatención.



La **autoatención cruzada seguida de concatenación** (figura 24) actúa en dos fases, una primera en que se aplica una autoatención cruzada y, posteriormente, se aplica una autoatención convencional sobre la concatenación de los vectores procedentes de la operación original.

Figura 24. Atención cruzada seguida de concatenación. La información procedente de dos canales distintos se procesa en la misma capa de autoatención y, posteriormente, es combinada en un canal único.

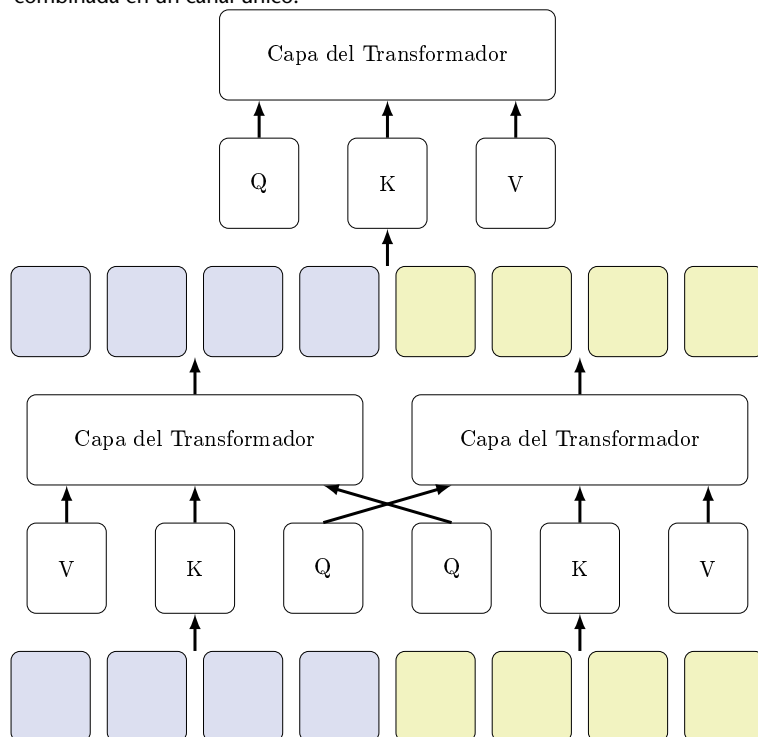


Tabla 4. Variantes de autoatención de interacción/fusión de canales multimodales. α y β son pesos. Att: Atención; $\mathcal{C}$: Concatenación; Tfs: Capas de transformador. $N_{(A)}$ y $N_{(B)}$ longitudes de secuencia de dos canales A y B de distinto modo. $MHSA(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ Función de autoatención multicabezal

autoatención	Formulación	Referencias
Suma temprana	$\mathbf{Z} \leftarrow Tf(\alpha \mathbf{Z}_{(A)} \oplus \beta \mathbf{Z}_{(B)})$	(Gavrilyuk y otros, 2020), (Xu y Zhu, 2021)
Concatenación temprana	$\mathbf{Z} \leftarrow Tf(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)}))$	(Sun y otros, 2019), (Guo y otros, 2020), (Shi y otros, 2022), (Zheng y otros, 2021a)
Jerárquica multi $\rightarrow$ monocanal	$\mathbf{Z} \leftarrow Tf_3(\mathcal{C}(Tf_1(\mathbf{Z}_{(A)}), Tf_2(\mathbf{Z}_{(B)})))$	(Li y otros, 2021)
Jerárquica mono $\rightarrow$ multicanal	$\begin{cases} \mathbf{Z}_{(A)} \leftarrow Tf_2(Tf_1(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)}))) \\ \mathbf{Z}_{(B)} \leftarrow Tf_3(Tf_1(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)}))) \end{cases}$	(Lin y otros, 2020)
Cruzada	$\begin{cases} \mathbf{Z}_{(A)} \leftarrow MHSA(\mathbf{Q}_B, \mathbf{K}_A, \mathbf{V}_A) \\ \mathbf{Z}_{(B)} \leftarrow MHSA(\mathbf{Q}_A, \mathbf{K}_B, \mathbf{V}_B) \end{cases}$	(Lu y otros, 2019), (Yun y otros, 2021)
Cruzada más concatenación	$\begin{cases} \mathbf{Z}_{(A)} \leftarrow MHSA(\mathbf{Q}_B, \mathbf{K}_A, \mathbf{V}_A) \\ \mathbf{Z}_{(B)} \leftarrow MHSA(\mathbf{Q}_A, \mathbf{K}_B, \mathbf{V}_B) \\ \mathbf{Z} \leftarrow Tf(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)})) \end{cases}$	(Hasan y otros, 2021), (Zhan y otros, 2021), (Tsai y otros, 2019)

2.7.3 Arquitecturas multimodales

El objetivo de estas estrategias es la consecución de una representación interna que integre de la manera más adecuada posible la información relevante procedente de cada uno de los modos de entrada. El sistema de autoatención escogido determina las características de la arquitectura. Así, hablamos de arquitecturas de **canal único**, **multicanal** o **híbridas**. Las arquitecturas de **canal único** son las derivadas del uso de los sistemas de autoatención de **suma** y **concatenación temprana** y disponen de un canal de tratamiento único de los datos. Las arquitecturas **multicanal** son las derivadas del uso de los sistemas de autoatención **cruzada** y dan lugar a canales múltiples de tratamiento diferenciado de los datos. Finalmente, las arquitecturas **híbridas**, derivadas de sistemas de autoatención **jerárquica** y **cruzada con concatenación** combinan partes de las dos anteriores.

2.8 Aplicaciones de los transformadores multimodales

En este subpartado podremos ver algunos de los trabajos publicados hasta el momento que utilizan transformadores multimodales.

- **MMBT** (Kiela y otros, 2019) es una arquitectura multimodal que integra imagen y texto para obtener una representación interna que puede ser utilizada para realizar tareas de clasificación.
- **VideoBERT** (Sun y otros, 2019) es una arquitectura multimodal con una finalidad similar a BERT pero utilizando vídeo como fuente de datos.
- **Med-BERT** (Rasmy y otros, 2021) es una arquitectura multimodal que utiliza registros de salud estructurados para representar la salud de pacientes y a partir de la misma poder predecir enfermedades.

- **Gato** (Reed y otros, 2022) es una arquitectura multimodal con un objeto generalista. A partir de datos procedentes de diversas fuentes (imagen, texto y acciones propias de un robot) es capaz de resolver hasta 604 tareas distintas, entre las cuales se encuentran jugar a multitud de juegos de Atari, hacer de chatbot, comandar un brazo articulado, etiquetar el contenido de una imagen, entre otras muchas. Todo ello utilizando una única arquitectura basada en transformadores. Es la primera aplicación generalista que integra imagen, procesamiento del lenguaje natural, aprendizaje por refuerzo y control de robots en un único agente.

Resumen

En este módulo hemos presentado la arquitectura de red neuronal de transformadores, la cual tiene la capacidad de reemplazar a arquitecturas especializadas como redes convolucionales, recurrentes y de grafos. Esta arquitectura no solo puede funcionar en el campo específico del procesamiento del lenguaje natural, para el cual fue diseñada originalmente, sino que también ha demostrado funcionar al máximo nivel para el tratamiento de otro tipo de señales específicas, así como para la resolución de problemas que combinan datos de distintas fuentes, lo que la convierte en un agente generalista para la resolución de problemas.

La arquitectura de los transformadores ha demostrado funcionar muy bien en áreas como la visión por computador, el procesamiento del lenguaje natural, el tratamiento de señales de audio, la predicción de series temporales y el aprendizaje por refuerzo, entre otras.

En el módulo se ha presentado una descripción rigurosa del algoritmo de la arquitectura y se han mostrado aplicaciones en distintas áreas, desde su aplicación de origen en el procesamiento del lenguaje natural, el procesamiento de imágenes y sonidos y hasta su uso en aplicaciones multimodales.

Conocer los detalles de implementación de la arquitectura de los transformadores es de gran importancia, ya que puede ser una solución general para la mayoría de los problemas de ciencia de datos.

Glosario

atención *f* mecanismo mediante el cual es posible enfatizar la importancia de una parte de la información presente en un vector multidimensional, representación interna de un estado.

embedding representación numérica de un *token*. Suele ser aprendida mediante una transformación que puede ser lineal o no lineal a partir de la codificación del *token* en un *vector one-hot*.

NLP siglas en inglés de *Natural Language Processing*, en castellano, ‘Procesamiento del Lenguaje Natural’.

NLU siglas en inglés de *Natural Language Understanding*, en castellano, ‘Comprensión del Lenguaje Natural’.

tokenización *f* subdivisión de los elementos de una secuencia de entrada en subelementos más pequeños. Por ejemplo, una frase en palabras, caracteres, agrupaciones de caracteres comunes, etc. A cada elemento constituyente se le llama *token*.

transformador *m* arquitectura de red neuronal que está formada por un conjunto de capas que integran en cada una de ellas mecanismos de atención.

vector one-hot *m* vector representativo del valor de un *token*. Cada *token* distinto se asocia a una posición diferente atendiendo a su correspondencia con un diccionario. El *token* queda representado por un vector ortogonal que tiene un uno en la posición que le corresponde dentro del diccionario y ceros en todas las demás.

Notación

La notación matemática de este módulo es la misma que la utilizada en Phuong y Hutter (2022). A efectos prácticos la reproducimos a continuación.

Símbolo	Tipo	Explicación
$[N]$	$:= \{1, \dots, N\}$	conjunto de enteros $1, 2, \dots, N - 1, N$
i, j	$\in \mathbb{N}$	índices enteros genéricos
V	$\cong [N_V]$	vocabulario
N_V	$\in \mathbb{N}$	tamaño del vocabulario
V^*	$= \bigcup_{\ell=0}^{\infty} V^\ell$	conjunto de secuencias de <i>tokens</i> ; por ejemplo, palabras y documentos
ℓ_{max}	$\in \mathbb{N}$	longitud máxima de la secuencia de <i>tokens</i>
ℓ	$\in [\ell_{max}]$	longitud de la secuencia de <i>tokens</i>
t	$\in [\ell]$	índice del token dentro de una secuencia
$d \dots$	$\in \mathbb{N}$	dimensión de varios vectores
$\mathbf{x}$	$\equiv x[1 : \ell]$	$\equiv x[1]x[2] \dots x[\ell] \in V^\ell$ secuencia de <i>tokens</i> primaria
$\mathbf{z}$	$\equiv z[1 : \ell]$	$\equiv z[1]z[2] \dots z[\ell] \in V^\ell$ secuencia de <i>tokens</i> de contexto
$M[i, j]$	$\in \mathbb{R}$	entrada $M_{i,j}$ de la matriz $M \in \mathbb{R}^{d \times d'}$
$M[i, :] \equiv M[i]$	$\in \mathbb{R}^{d'}$	fila i de la matriz $M \in \mathbb{R}^{d \times d'}$
$M[:, j]$	$\in \mathbb{R}^d$	columna j de la matriz $M \in \mathbb{R}^{d \times d'}$
$\mathbf{e}$	$\mathbb{R}^{d_e}$	representación vectorial / representación vectorial (embedding) de un token
$\mathbf{X}$	$\mathbb{R}^{d_e \times \ell_{\mathcal{X}}}$	codificación de la secuencia de tokens primaria
$\mathbf{Z}$	$\mathbb{R}^{d_e \times \ell_{\mathcal{Z}}}$	codificación de la secuencia de tokens de contexto
Mask	$\mathbb{R}^{\ell_{\mathcal{Z}} \times \ell_{\mathcal{X}}}$	matriz de enmascarado, determina el contexto de atención para cada <i>token</i>
L, L_{enc}, L_{dec}	$\mathbb{N}$	número de capas de red (codificación y decodificación)
l	$\in [L]$	índice de la capa de la red
H	$\mathbb{N}$	número de cabezales de atención
h	$\in [H]$	índice del cabezal de atención
N_{data}	$\in \mathbb{N}$	(i.i.d.) tamaño de la muestra
n	$\in [N_{data}]$	índice de la secuencia de muestra
η	$\in (0, \infty)$	ratio de aprendizaje
τ	$\in (0, \infty)$	temperatura, controla la diversidad en tiempo de inferencia
$\mathbf{W}_e$	$\in \mathbb{R}^{d_e \times N_V}$	matriz de <i>embedding</i> de los tokens
$\mathbf{W}_p$	$\in \mathbb{R}^{d_e \times \ell_{max}}$	matriz de <i>embedding</i> de posición
$\mathbf{W}_u$	$\in \mathbb{R}^{N_V \times d_e}$	matriz de conversión de <i>embedding</i> a <i>token</i>
$\mathbf{W}_q$	$\in \mathbb{R}^{d_{attn} \times d_{\mathcal{X}}}$	parámetros de la matriz de consulta
$\mathbf{b}_q$	$\in \mathbb{R}^{d_{attn}}$	sesgo de consulta
$\mathbf{W}_k$	$\in \mathbb{R}^{d_{attn} \times d_{\mathcal{Z}}}$	parámetros de la matriz de clave
$\mathbf{b}_k$	$\in \mathbb{R}^{d_{attn}}$	sesgo de clave
$\mathbf{W}_v$	$\in \mathbb{R}^{d_{out} \times d_{\mathcal{Z}}}$	parámetros de la matriz de valor
$\mathbf{b}_v$	$\in \mathbb{R}^{d_{attn}}$	sesgo de valor
$\mathbf{W}_{qkv}$		colección de parámetros de una capa de atención
$\mathbf{W}_o$	$\in \mathbb{R}^{d_{out} \times Hd_{mid}}$	parámetros de la matriz de salida
$\mathbf{b}_o$	$\in \mathbb{R}^{d_{out}}$	sesgo de salida
$\mathbf{W}$		colección de parámetros de una capa de multiatención
$\mathbf{W}_{mlp}$	$\in \mathbb{R}^{d_1 \times d_2}$	parámetros de una MLP del transformador
$\mathbf{b}_{mlp}$	$\in \mathbb{R}^{d_1}$	sesgo correspondiente a una MLP del transformador
γ	$\in \mathbb{R}^{d_e}$	parámetro de escalado de la capa de normalización
ϵ	$\in \mathbb{R}^{d_e}$	parámetro de offset de la capa de normalización
$\mathbf{W}, \mathbf{b}$	$\in \mathbb{R}^d$	colección de todos los parámetros del transformador

Bibliografía

«Top 1 Accuracy in ImageNet Image Classification». [Fecha de consulta: 21 de septiembre de 2022]. Disponible en: <<https://paperswithcode.com/sota/image-classification-on-imagenet>>.

Akbari, H.; Yuan, L.; Qian, R.; Chuang, W.-H.; Chang, S.-F.; Cui, Y.; Gong, B. (2021). «Vatt: Transformers for multimodal self-supervised learning from raw video, audio and text». *Advances in Neural Information Processing Systems* (vol. 34, págs. 24206–24221).

Arnab, A.; Dehghani, M.; Heigold, G.; Sun, C.; Lučić, M.; Schmid, C. (2021). «Vivit: A video vision transformer». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 6836–6846).

Baevski, A.; Zhou, Y.; Mohamed, A.; Auli, M. (2020). «wav2vec 2.0: A framework for self-supervised learning of speech representations». *Advances in Neural Information Processing Systems* (vol. 33, págs. 12449–12460).

Bahdanau, D.; Cho, K.; Bengio, Y. (2014). «Neural machine translation by jointly learning to align and translate». *arXiv preprint arXiv:1409.0473*.

Bracewell, R. N.; Bracewell, R. N. (1986). *The Fourier transform and its applications* (vol. 31999). Nueva York: McGraw-Hill.

Bronstein, M. M.; Bruna, J.; Cohen, T.; Velicković, P. (2021). «Geometric deep learning: Grids, groups, graphs, geodesics, and gauges». *arXiv preprint arXiv:2104.13478*.

Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; Zagoruyko, S. (2020). «End-to-end object detection with transformers». En: «European conference on computer vision» (págs. 213–229). Springer.

Chen, H.; Wang, Y.; Guo, T.; Xu, C.; Deng, Y.; Liu, Z.; Ma, S.; Xu, C.; Xu, C.; Gao, W. (2021a). «Pre-trained image processing transformer». En: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (págs. 12299–12310).

Chen, T.; Saxena, S.; Li, L.; Fleet, D. J.; Hinton, G. (2021b). «Pix2seq: A language modeling framework for object detection». *arXiv preprint arXiv:2109.10852*.

Cheng, B.; Misra, I.; Schwing, A. G.; Kirillov, A.; Girdhar, R. (2022). «Masked-attention mask transformer for universal image segmentation». En: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (págs. 1290–1299).

Cheng, B.; Schwing, A.; Kirillov, A. (2021). «Per-pixel classification is not all you need for semantic segmentation». *Advances in Neural Information Processing Systems* (vol. 34, págs. 17864–17875).

Dai, X.; Chen, Y.; Yang, J.; Zhang, P.; Yuan, L.; Zhang, L. (2021). «Dynamic detr: End-to-end object detection with dynamic attention». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 2988–2997).

Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. (2018). «Bert: Pre-training of deep bidirectional transformers for language understanding». *arXiv preprint arXiv:1810.04805*.

Ding, M.; Xiao, B.; Codella, N.; Luo, P.; Wang, J.; Yuan, L. (2022). «DaViT: Dual Attention Vision Transformers». *arXiv preprint arXiv:2204.03645*.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S. y otros (2020). «An image is worth 16x16 words: Transformers for image recognition at scale». *arXiv preprint arXiv:2010.11929*.

Gage, P. (1994). «A new algorithm for data compression». *C Users Journal* (vol. 12, n.º 2, págs. 23–38).

Gao, P.; Zheng, M.; Wang, X.; Dai, J.; Li, H. (2021). «Fast convergence of DETR with spatially modulated co-attention». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 3621–3630).

- Gavrilyuk, K.; Sanford, R.; Javan, M.; Snoek, C. G.** (2020). «Actor-transformers for group activity recognition». En: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (págs. 839–848).
- Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S. y otros** (2020). «Graphcodebert: Pre-training code representations with data flow». *arXiv preprint arXiv:2009.08366*.
- Hasan, M. K.; Lee, S.; Rahman, W.; Zadeh, A.; Mihalcea, R.; Morency, L.-P.; Hoque, E.** (2021). «Humor knowledge enriched transformer for understanding multimodal humor». En: «Proceedings of the AAAI Conference on Artificial Intelligence» (vol. 35, págs. 12972–12980).
- He, K.; Gkioxari, G.; Dollár, P.; Girshick, R.** (2017). «Mask R-CNN». En: «Proceedings of the IEEE international conference on computer vision» (págs. 2961–2969).
- He, K.; Zhang, X.; Ren, S.; Sun, J.** (2016). «Deep residual learning for image recognition». En: «Proceedings of the IEEE conference on computer vision and pattern recognition» (págs. 770–778).
- Hinton, G.; Vinyals, O.; Dean, J. y otros** (2015). «Distilling the knowledge in a neural network». *arXiv preprint arXiv:1503.02531* (vol. 2, n.º 7).
- Hochreiter, S.; Schmidhuber, J.** (1997). «Long short-term memory». *Neural computation* (vol. 9, n.º 8, págs. 1735–1780).
- Hornik, K.; Stinchcombe, M.; White, H.** (1989). «Multilayer feedforward networks are universal approximators». *Neural networks* (vol. 2, n.º 5, págs. 359–366).
- Howard, J.; Ruder, S.** (2018). «Universal language model fine-tuning for text classification». *arXiv preprint arXiv:1801.06146*.
- Ioannou, Y. A.** (2018). *Structural priors in deep neural networks*. Tesis Doctoral, University of Cambridge.
- Joulin, A.; Grave, E.; Bojanowski, P.; Douze, M.; Jégou, H.; Mikolov, T.** (2016). «FastText.zip: Compressing text classification models». *arXiv preprint arXiv:1612.03651*.
- Kiela, D.; Bhooshan, S.; Firooz, H.; Perez, E.; Testuggine, D.** (2019). «Supervised multimodal bitransformers for classifying images and text». *arXiv preprint arXiv:1909.02950*.
- Kudo, T.** (2018). «Subword regularization: Improving neural network translation models with multiple subword candidates». *arXiv preprint arXiv:1804.10959*.
- Kumar, M.; Weissenborn, D.; Kalchbrenner, N.** (2021). «Colorization transformer». *arXiv preprint arXiv:2102.04432*.
- Li, R.; Yang, S.; Ross, D. A.; Kanazawa, A.** (2021). «Ai choreographer: Music conditioned 3D dance generation with AIST++». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 13401–13412).
- Lin, J.; Yang, A.; Zhang, Y.; Liu, J.; Zhou, J.; Yang, H.** (2020). «Interbert: Vision-and-language interaction for multi-modal pretraining». *arXiv preprint arXiv:2003.13198*.
- Lin, T.-Y.; Dollár, P.; Girshick, R.; He, K.; Hariharan, B.; Belongie, S.** (2017a). «Feature pyramid networks for object detection». En: «Proceedings of the IEEE conference on computer vision and pattern recognition» (págs. 2117–2125).
- Lin, T.-Y.; Goyal, P.; Girshick, R.; He, K.; Dollár, P.** (2017b). «Focal loss for dense object detection». En: «Proceedings of the IEEE international conference on computer vision» (págs. 2980–2988).
- Lin, Y.-B.; Tseng, H.-Y.; Lee, H.-Y.; Lin, Y.-Y.; Yang, M.-H.** (2021). «Exploring cross-video and cross-modality signals for weakly-supervised audio-visual video parsing». *Advances in Neural Information Processing Systems* (vol. 34, págs. 11449–11461).
- Liu, S.; Li, F.; Zhang, H.; Yang, X.; Qi, X.; Su, H.; Zhu, J.; Zhang, L.** (2022). «DAB-DETR: Dynamic anchor boxes are better queries for DETR». *arXiv preprint arXiv:2201.12329*.
- Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V.** (2019). «Roberta: A robustly optimized bert pretraining approach». *arXiv preprint arXiv:1907.11692*.
- Liu, Y.; Zhang, Y.; Wang, Y.; Hou, F.; Yuan, J.; Tian, J.; Zhang, Y.; Shi, Z.; Fan, J.; He, Z.** (2021). «A survey of visual transformers». *arXiv preprint arXiv:2111.06091*.

- Lu, J.; Batra, D.; Parikh, D.; Lee, S.** (2019). «Vilbert: Pretraining task-agnostic visio-linguistic representations for vision-and-language tasks». *Advances in neural information processing systems* (vol. 32).
- Meng, D.; Chen, X.; Fan, Z.; Zeng, G.; Li, H.; Yuan, Y.; Sun, L.; Wang, J.** (2021). «Conditional detr for fast training convergence». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 3651–3660).
- Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G. S.; Dean, J.** (2013). «Distributed Representations of Words and Phrases and their Compositionality». En: C. Burges; L. Bottou; M. Welling; Z. Ghahramani; K. Weinberger; Curran Associates, Inc. (eds.) «Advances in Neural Information Processing Systems» (vol. 26). Disponible en: <<https://proceedings.neurips.cc/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf>>.
- Nagrani, A.; Yang, S.; Arnab, A.; Jansen, A.; Schmid, C.; Sun, C.** (2021). «Attention bottlenecks for multimodal fusion». *Advances in Neural Information Processing Systems* (vol. 34, págs. 14200–14213).
- Nussbaumer, H. J.** (1981). «The Fast Fourier Transform». En: «Fast Fourier Transform and Convolution Algorithms» (págs. 80–111). Springer.
- Pennington, J.; Socher, R.; Manning, C. D.** (2014). «Glove: Global vectors for word representation». En: «Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)» (págs. 1532–1543).
- Pham, H.; Dai, Z.; Xie, Q.; Le, Q. V.** (2021). «Meta pseudo labels». En: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (págs. 11557–11568).
- Phuong, M.; Hutter, M.** (2022). «Formal Algorithms for Transformers». *arXiv preprint arXiv:2207.09238*.
- Picek, L.; Šulc, M.; Matas, J.; Heilmann-Clausen, J.; Jeppesen, T. S.; Lind, E.** (2022). «Automatic fungi recognition: Deep learning meets mycology». *Sensors* (vol. 22, n.º 2, pág. 633).
- Press, O.; Wolf, L.** (2016). «Using the output embedding to improve language models». *arXiv preprint arXiv:1608.05859*.
- Radford, A.; Kim, J. W.; Xu, T.; Brockman, G.; McLeavey, C.; Sutskever, I.** (2022). «Robust Speech Recognition via Large-Scale Weak Supervision».
- Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I.** (2018). «Improving language understanding with unsupervised learning».
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I. y otros** (2019). «Language models are unsupervised multitask learners». *OpenAI blog* (vol. 1, n.º 8, pág. 9).
- Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P. J. y otros** (2020). «Exploring the limits of transfer learning with a unified text-to-text transformer». *J. Mach. Learn. Res.* (vol. 21, n.º 140, págs. 1–67).
- Rasmy, L.; Xiang, Y.; Xie, Z.; Tao, C.; Zhi, D.** (2021). «Med-BERT: pretrained contextualized embeddings on large-scale structured electronic health records for disease prediction». *NPJ digital medicine* (vol. 4, n.º 1, págs. 1–13).
- Reed, S.; Zolna, K.; Parisotto, E.; Colmenarejo, S. G.; Novikov, A.; Barth-Maron, G.; Gimenez, M.; Sulsky, Y.; Kay, J.; Springenberg, J. T. y otros** (2022). «A generalist agent». *arXiv preprint arXiv:2205.06175*.
- Ronneberger, O.; Fischer, P.; Brox, T.** (2015). «U-net: Convolutional networks for biomedical image segmentation». En: «International Conference on Medical image computing and computer-assisted intervention» (págs. 234–241). Springer.
- Shi, B.; Hsu, W.-N.; Lakhota, K.; Mohamed, A.** (2022). «Learning audio-visual speech representation by masked multimodal cluster prediction». *arXiv preprint arXiv:2201.02184*.
- Stevens, S. S.; Volkman, J.; Newman, E. B.** (1937). «A scale for the measurement of the psychological magnitude pitch». *The Journal of the Acoustical Society of America* (vol. 8, n.º 3, págs. 185–190).
- Stewart, R.; Andriluka, M.; Ng, A. Y.** (2016). «End-to-end people detection in crowded scenes». En: «Proceedings of the IEEE conference on computer vision and pattern recognition» (págs. 2325–2333).

- Subakan, C.; Ravanelli, M.; Cornell, S.; Bronzi, M.; Zhong, J.** (2021). «Attention is all you need in speech separation». En: «ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)» (págs. 21–25). IEEE.
- Sun, C.; Myers, A.; Vondrick, C.; Murphy, K.; Schmid, C.** (2019). «Videobert: A joint model for video and language representation learning». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 7464–7473).
- Touvron, H.; Cord, M.; Douze, M.; Massa, F.; Sablayrolles, A.; Jégou, H.** (2021). «Training data-efficient image transformers & distillation through attention». En: «International Conference on Machine Learning» (págs. 10.347–10.357). PMLR.
- Tsai, Y.-H. H.; Bai, S.; Liang, P. P.; Kolter, J. Z.; Morency, L.-P.; Salakhutdinov, R.** (2019). «Multimodal transformer for unaligned multimodal language sequences». En: «Proceedings of the conference. Association for Computational Linguistics. Meeting» (vol. 2019, pág. 6558). NIH Public Access.
- Tuli, S.; Dasgupta, I.; Grant, E.; Griffiths, T. L.** (2021). «Are Convolutional Neural Networks or Transformers more like human vision?» *arXiv preprint arXiv:2105.07197*.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; Polosukhin, I.** (2017). «Attention is all you need». *Advances in neural information processing systems* (vol. 30).
- Wang, W.; Xie, E.; Li, X.; Fan, D.-P.; Song, K.; Liang, D.; Lu, T.; Luo, P.; Shao, L.** (2021). «Pyramid vision transformer: A versatile backbone for dense prediction without convolutions». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 568–578).
- Wang, Y.; Zhang, X.; Yang, T.; Sun, J.** (2022). «Anchor detr: Query design for transformer-based detector». En: «Proceedings of the AAAI Conference on Artificial Intelligence» (vol. 36, págs. 2567–2575).
- Weng, Z.; Yang, X.; Li, A.; Wu, Z.; Jiang, Y.-G.** (2021). «Semi-supervised vision transformers». *arXiv preprint arXiv:2111.11067*.
- Wu, B.; Xu, C.; Dai, X.; Wan, A.; Zhang, P.; Yan, Z.; Tomizuka, M.; Gonzalez, J.; Keutzer, K.; Vajda, P.** (2020). «Visual transformers: Token-based image representation and processing for computer vision». *arXiv preprint arXiv:2006.03677*.
- Xu, P.; Zhu, X.** (2021). «Deepchange: A large long-term person re-identification benchmark with clothes change». *arXiv e-prints* (págs. arXiv–2105).
- Xu, P.; Zhu, X.; Clifton, D. A.** (2022). «Multimodal Learning with Transformers: A Survey». *arXiv preprint arXiv:2206.06488*.
- Yao, Z.; Ai, J.; Li, B.; Zhang, C.** (2021). «Efficient DETR: improving end-to-end object detector with dense prior». *arXiv preprint arXiv:2104.01318*.
- Yuan, L.; Chen, Y.; Wang, T.; Yu, W.; Shi, Y.; Jiang, Z.-H.; Tay, F. E.; Feng, J.; Yan, S.** (2021). «Tokens-to-token ViT: Training vision transformers from scratch on imagenet». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 558–567).
- Yun, H.; Yu, Y.; Yang, W.; Lee, K.; Kim, G.** (2021). «Pano-AVQA: Grounded audio-visual question answering on 360deg videos». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 2031–2041).
- Zhan, X.; Wu, Y.; Dong, X.; Wei, Y.; Lu, M.; Zhang, Y.; Xu, H.; Liang, X.** (2021). «Product1M: Towards weakly supervised instance-level product retrieval via cross-modal pretraining». En: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (págs. 11782–11791).
- Zheng, R.; Chen, J.; Ma, M.; Huang, L.** (2021a). «Fused acoustic and text encoding for multimodal bilingual pretraining and speech translation». En: «International Conference on Machine Learning» (págs. 12736–12746). PMLR.
- Zheng, S.; Lu, J.; Zhao, H.; Zhu, X.; Luo, Z.; Wang, Y.; Fu, Y.; Feng, J.; Xiang, T.; Torr, P. H. y otros** (2021b). «Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers». En: «Proceedings of the IEEE/CVF conference on computer vision and pattern recognition» (págs. 6881–6890).
- Zhu, X.; Su, W.; Lu, L.; Li, B.; Wang, X.; Dai, J.** (2020). «Deformable DETR: Deformable transformers for end-to-end object detection». *arXiv preprint arXiv:2010.04159*.